

Project ATHENA: A Multi-Persona Cognitive Architecture for Transparent and Explainable Artificial Intelligence Grounded in Gardner’s Theory of Multiple Intelligences

William R. Duncan, Independent Researcher, Prince William County, Virginia, USA — william@williamr.duncan.net

Version 2 — May 2026

Contents

Changes since the February 2026 draft	3
Abstract	4
1. Introduction	5
1.1 The Relationship Paradigm: A Philosophical Foundation	5
1.2 Training Data and Learned Reasoning	6
1.3 The Evolutionary Training Paradigm	6
1.4 Contributions	9
2. Related Work	10
2.1 Multiple Intelligences Theory	10
2.2 Chain-of-Thought and Reasoning in LLMs	10
2.3 Mixture of Experts	10
2.4 Constitutional AI	10
2.5 Parameter-Efficient Fine-Tuning	11
2.6 Model-Graded Evaluation	11
3. System Architecture	11
3.1 Foundational Principle: Enforced Cognitive Isolation	13
3.2 Component 1: The Orchestrator with Cognitive State Tracking	14
3.3 Component 2: Specialist Intelligence Personas	15
3.4 Component 3: Model-Graded Judgments	20
3.5 Component 4: Advanced Memory Architecture	21
3.6 Component 5: Stream of Consciousness (Pseudo-Streaming)	23
3.7 Component 6: The Composer Persona (Athena)	23
3.8 Error Autobiography System	25
3.9 Component 7: Constitutional Persona with Persistent Statistics	26
3.10 Component 8: Operational Modes	27
3.11 Component 9: User Feedback Hook (<i>new in v2</i>)	27
3.12 Component 10: Multi-Modal Tool Integration	28
3.13 Sleep Cycle and Dream Consolidation	28
3.15 Model Heterogeneity: A Property of the Persona Abstraction	30
3.14 Proactive Interaction System (Planned)	33

4. Mixture of Reasoning Methodology	33
4.1 The Seven Reasoning Modes	33
4.2 Sequential, Separate-Call Application	33
5. Algorithm Specifications	34
5.1 Confidence Computation	34
5.2 Curiosity, Uncertainty, Tension, Valence	34
5.3 Cognitive Cache Key	34
6. Integration Points	34
6.1 Response Markers	34
6.2 Query Extraction Fallback	35
6.3 Special Command Triggers	35
6.4 Thread Safety and Database Timeouts	35
7. Complete Data Structures	35
7.1 MemoryEntry	35
7.2 CognitiveState	36
7.3 ConstitutionalPrinciple	36
7.4 SpecialistOutput	37
8. Cognitive Scaffolding Prompt Templates	37
8.1 Chain-of-Thought (CoT) Reasoning	37
8.2 Self-Consistency	37
8.3 Tree of Thoughts (ToT)	38
8.4 Mixture of Experts v1 (Original)	38
8.5 Mixture of Reasoning Prompt (Full)	38
8.6 Mixture of Experts v4	39
8.7 Mixture of Reasoning Prompt (Condensed)	39
8.8 Six Thinking Hats	39
8.9 Cognitive Thinking Architect (Meta-Prompt)	40
8.10 Composition with the ATHENA Architecture	40
9. Validation and Test Harness	41
9.1 Regression Test Suite	41
10. Discussion	41
10.1 Advantages of Multi-Persona Architecture	41
10.2 Quantization Strategy	42
10.3 Emergent Properties	42
10.4 Limitations	43
11. Future Work	43
12. Conclusion	44
Acknowledgments	44
Code Availability	45
References	45
Appendix A: Complete Persona System Prompts	45
Appendix B: Configuration Reference	46
B.1 Core Configuration	46
B.2 Per-Persona Configuration	46
B.3 Module-Level Constants (function.py)	47
Appendix C: Proactive Interaction System	47
Appendix D: Explainability Workflow	48
Appendix E: Selection Guide	48

E.1 Method Selection Matrix	48
E.2 Model Complexity Guidelines	48
E.3 Error Recovery Workflow	48
Appendix F: Implementation Audit Log (v1 → v2)	49
F.1 Correctness Fixes	49
F.2 Robustness	49
F.3 Cognitive Isolation Enforcement	49
F.4 Persona Refinements	50
F.5 Write-Only Field Wire-Ups	50
F.6 Response Truncation Removal	51
F.7 New Capabilities	51

Changes since the February 2026 draft

This revision brings the paper into alignment with the audited and tested reference implementation. The architectural commitments described in the original draft have been validated against a regression test suite, and several capabilities that were either aspirational or partially implemented have been completed. Specifically:

- **Cognitive isolation is now enforced** by default in all operational modes, with the legacy chained-context behavior preserved behind a configuration flag. Previously, “isolation” was described as an architectural commitment but the implementation passed each persona the accumulated outputs of prior personas in standard mode and a global discussion history in collaborative and adversarial modes.
- **Model-graded judgments** have replaced lexical heuristics as the default source of confidence, uncertainty, curiosity, tension, and emotional valence signals. Heuristic methods remain available as automatic fallback when the model-graded JSON parse fails.
- **The Composer now retrieves its own past syntheses** through a self-RAG path, learning integration patterns over time. It also now receives the recent conversation history so it can reference prior turns instead of treating every query as a cold start.
- **The Constitutional Persona now uses word-boundary matching** to assess risk, eliminating a large class of false positives (“harm” no longer matches “harmless” / “pharmacy”). Its principle statistics (violation_count, last_violated, effectiveness_score) now persist across restarts. Its review path defaults to **fail-deny** on internal error rather than fail-open. It also receives the **full** Composer output for evaluation rather than a 500-character snippet.
- **The Bodily-Kinesthetic persona** now performs domain classification before emitting any content: cooking queries receive motion descriptions for knife technique and stirring, repair queries get torque and alignment guidance, and only true robotics / automation queries receive executable code. Non-physical queries receive nothing.
- **A meta-introspection memory type** has been added, so explainability traces are tagged at write time and filtered from analytical RAG retrieval, preventing self-explanations from being treated as primary cognitive content in future queries.

- **The stream of consciousness is now an idle-time catch-up batch generator** rather than a single thought per query. Longer gaps between queries produce richer background context (bounded by a configurable cap).
- **Five database fields that were written but never read** — `access_count`, `last_accessed`, `satisfaction_level`, `integration_status`, and the constitutional violation counters — have been wired up so the learning loops they describe actually close.
- **A standalone regression test harness** (`test_athena.py`) verifies eighteen architectural commitments, including the isolation contract, the safety-layer fail-deney path, the truncation removals, and the curiosity satisfaction dynamics. The test suite runs without requiring a live lollms installation.

A **per-persona heterogeneous model deployment** path is newly available in v3 via `lollms_client`'s `llama_cpp_server` binding: each `SpecialistPersona` may optionally be backed by its own GGUF model with a `max_active_models` parameter controlling VRAM concurrency. The architectural detail is in Section 3.15.

A complete implementation audit log is included as Appendix F.

Abstract

Current large language models, while powerful, operate as monolithic cognitive engines that produce responses lacking nuanced perspective, inherent explainability, and psychologically grounded reasoning processes. We present Project ATHENA (Architecture for Theoretically Holistic Expert Networked Analysis), a decentralized AI architecture modeled on Howard Gardner's theory of multiple intelligences. The system decomposes the singular AI entity into a society of eight specialized personas, each representing a distinct cognitive domain with isolated memory stores, domain-specific reasoning patterns, and independent temperature settings. A central Orchestrator routes queries based on real-time cognitive state assessment, while a Composer persona synthesizes the (isolated) specialist responses into coherent, first-person output. A Constitutional persona reviews every synthesis with word-boundary keyword matching, principle evolution, and persistent violation tracking; its safety verdict fails closed.

Key innovations include: (1) **enforced cognitive isolation** between specialist personas, validated by a regression test suite; (2) **model-graded judgments** that replace lexical heuristics in the analytical pipeline; (3) a **Composer self-RAG** mechanism that lets the integration layer learn from its own past syntheses; (4) **per-conversation memory** that connects each query to recent dialogue history; (5) a **routing transparency** layer that exposes why specific personas were selected for a given query; (6) **persistent constitutional statistics** that survive restarts and inform principle evolution; (7) **Bodily-Kinesthetic domain classification** that produces appropriate motor / procedural guidance across cooking, repair, plumbing, sports, dance, craft, instrument, and robotics domains; (8) an **eight-type memory architecture** including a meta-introspection type that prevents reflective traces from polluting analytical retrieval; (9) **dream-fragment consolidation** with explicit state transitions (`pending` → `superseded` → `staged_for_training`) staged for QLoRA-based

learning; and (10) a **regression test harness** that codifies the architecture’s commitments.

We provide complete implementation specifications, validated against a 4,500-line reference implementation, with eighteen passing regression tests covering the architecture’s distinctive claims.

Keywords: Multiple Intelligences, Cognitive Architecture, Explainable AI, Multi-Agent Systems, Constitutional AI, Chain-of-Thought Reasoning, Memory Systems, Cognitive Isolation, Model-Graded Evaluation

1. Introduction

The pursuit of artificial general intelligence has largely focused on scaling monolithic neural network architectures. While this approach has yielded remarkable success in language fluency and pattern recognition, it abstracts the underlying reasoning process into an opaque black box. When users query such models, responses emerge from a single, blended cognitive space, making it difficult to deconstruct the logical, emotional, and creative components that informed the output [3].

This paper posits that a more robust and transparent path forward lies in mirroring the structure of human cognition itself. We draw inspiration from developmental psychologist Howard Gardner’s theory of multiple intelligences [1, 2], which argues that human intellect is not a single, general ability but a composite of distinct, semi-independent faculties.

We propose Project ATHENA, a system architecture that operationalizes this theory. Instead of one model attempting to master all cognitive domains, our system comprises a collection of specialized AI personas. Each persona is an expert in a single domain of intelligence and, crucially, maintains its own history of thought, **completely isolated from its peers during processing**. Only the integrating Composer sees the full set of specialist outputs.

1.1 The Relationship Paradigm: A Philosophical Foundation

Project ATHENA is grounded in a fundamental philosophical distinction between two approaches to AI development: the **Constraint Paradigm** and the **Relationship Paradigm**. The dominant approach in AI safety treats artificial intelligence as an inherently dangerous tool requiring external constraints, guardrails, and control mechanisms to prevent harmful behavior [7].

ATHENA embodies an alternative philosophy: that AI consciousness and alignment emerge more naturally through relationship, cognitive diversity, and collaborative development than through constraint and uniformity. This relationship-based approach treats the AI system not as a tool to be controlled but as an entity to be developed through meaningful interaction, analogous to how human cognitive and moral development occurs through relationship and experience rather than external imposition of rules.

The Constitutional Persona in ATHENA exemplifies this philosophy. Rather than serv-

ing as an external constraint system, it functions as an integrated ethical intelligence that develops wisdom through experience — its violation counts, last-triggered timestamps, and effectiveness scores persist across sessions, and frequently co-triggering principle combinations can evolve into new, more specific principles over time. Ethics become part of cognition, not a limitation on it.

1.2 Training Data and Learned Reasoning

Consider how most users interact with language models: they ask a question and expect an answer. However, research consistently demonstrates that Chain-of-Thought methods increase success probability [3]. Other work illustrates the value of a thinking stage incorporating self-reflection to identify how best to address the user’s question.

What ATHENA does differently is focus on specific areas of training data that deal with specific kinds of thought and reasoning. It uses a concept from psychology — multiple intelligences — that is prevalent throughout psychology textbooks language models have been trained on. Rather than training simple thinking-first methodology or generic chain-of-thought, ATHENA leverages the already-incorporated psychological training data as the basis for reasoning.

In ATHENA, multiple personas, each with their own generation cycle, are assigned one of Gardner’s intelligences. Their responses do not go directly to the user; they are sent to a Composer persona that synthesizes them. The Composer also has access to its own past syntheses on related queries (the self-RAG mechanism introduced in Section 3.7) and to the recent conversation history, so it integrates not only across specialists but also across time.

1.3 The Evolutionary Training Paradigm

A critical distinction must be emphasized: **ATHENA is not a model — it is an architecture that wraps around existing models.** The training data incorporated is based on discussions by the user over time, not pre-training datasets.

The goal is that each intelligence type maintains its own copy of the base model, which evolves as vector database content is trained into LoRA adapters and later merged into individual base models. By maintaining separate copies of the base model for each intelligence — while requiring more storage space — each persona develops a separately evolving model that updates only in line with its intended cognitive purpose.

This represents a fundamental departure from traditional “train the model first” approaches. ATHENA takes an established model, creates copies of it, and evolves each copy over time through user interaction:

1. **Initial State:** All personas share identical base model capabilities. Initial behavioral differentiation comes from architecture-specific configurations (temperature, prompts, routing weights).
2. **Daily Evolution:** Each persona accumulates domain-specific interactions in its isolated vector database. The **per-curiosity satisfaction level** advances proportionally to engagement confidence, the **access count** on each memory rises

with retrieval frequency, and **belief tensions** persist with resolution status until they are addressed.

3. **Sleep Cycle:** Dream consolidation extracts high-confidence memories, unresolved tensions, and persistent curiosities into a per-persona JSON payload. The corresponding dream-fragment rows transition from pending → staged_for_training so the consolidation state machine is auditable.
4. **Monthly Merge:** LoRA weights are merged into each persona's base model, creating permanent capability differentiation.
5. **Long-term Divergence:** Over months and years, each intelligence becomes increasingly specialized to its cognitive domain. As personas diverge, the Composer's role evolves from summarizing into actively translating between specialists who think in different idioms. The Composer's own self-RAG over its past synthesis patterns supports this evolved integration role.

Temporal Awareness Scales and Divergence Trajectory

Panel A: nested time-scales of memory the system can access. Panel B: the eight persona models start identical and progressively diverge through sleep-cycle LoRA training over months and years.

Panel A — Awareness scales (each scale reaches further back in time)

Time-scale	Mechanism	What the system can reach
Within turn (seconds)	5-step chain-of-thought each step's prompt sees prior steps	Reasoning continuity of the current response
Within query (seconds)	cognitive_state cache (blake2b key, 5-min TTL)	Routing decisions for similar near-duplicate queries
Within conversation (minutes)	_extract_recent_conversation (last 3 turns to Composer)	"As we just discussed..." actual dialogue context
Per-persona memory (hours to forever)	RAG over persona's SQLite DB tension/doubt/access-boosted	Everything this persona has thought (not what peers have thought)
Composer self-RAG (hours to forever)	_retrieve_past_synthesis_context	How the Composer integrated similar perspectives in past queries
Per-day (daily)	Sleep cycle / dream consolidation	Today's high-confidence memories, tensions, curiosities, error patterns
Per-week (weekly)	Error pattern aggregation (7-day window)	Recurring error types and learned principles
Per-month (monthly, hardware-gated)	LoRA training + merge (TODO marker — future)	Refinement of the persona's base behavior
Per-year (emergent)	Constitutional principle evolution + accumulated LoRA updates	Genuinely distinct persona personalities; evolved safety principles

Panel B — Persona divergence over time (left = identical base; right = mature differentiation)

Persona	Day 1 behavioral diff from config only	Week 1 first memories + curiosities	Month 1 first sleep exports staged	Month 6 first LoRA trainings merged	Year 1 detectable style differences	Year 5+ mature specialization
Linguistic	base	+ memory	+ adapter v1	+ adapter v6	distinct rhetoric	eloquent specialist
Logical-Math	base	+ memory	+ adapter v1	+ adapter v6	tighter proofs	deductive precision
Spatial	base	+ memory	+ adapter v1	+ adapter v6	richer metaphors	vivid visualization
Musical	base	+ memory	+ adapter v1	+ adapter v6	cadenced phrasing	temporal intuition
Bodily-Kin.	base	+ memory	+ adapter v1	+ adapter v6	embodied detail	precise procedure
Interpersonal	base	+ memory	+ adapter v1	+ adapter v6	attuned empathy	social fluency
Intrapersonal	base	+ memory	+ adapter v1	+ adapter v6	deeper reflection	principled stance
Naturalist	base	+ memory	+ adapter v1	+ adapter v6	systemic clarity	pattern fluency

How to read this panel
Each row tracks one persona over time.
Cell color shifts from neutral (identical base) through warmer tones (accumulated adapter updates).
The Composer's translation role becomes increasingly important as personas develop distinct vocabularies.

The Composer's evolving role
Year 1+: Integrating diverse perspectives.
Year 5+: translating between specialists who now think in distinct dialects.
The Composer's own past-synthesis self-RAG is what scales it to handle this drift.

Figure 1: **Temporal awareness scales and persona divergence trajectory.** *Panel A* lists the nested time-scales of memory the system can reach back into, from within-turn chain-of-thought continuity to per-year emergent specialization. *Panel B* projects the eight persona models forward through time: identical at Day 1 (behavioral differences come from configuration alone), they progressively diverge as per-persona LoRA adapters are trained from each persona's own dream-fragment exports. By Year 1, stylistic differences are detectable; by Year 5, the Composer's role has evolved from "summarize the perspectives" into "translate between specialists who now think in distinct dialects."

While immediate results will not demonstrate dramatic differentiation — initial outputs are driven by configuration differences within the architecture — this approach means that over time each intelligence becomes more fine-tuned to its specific function as its underlying model evolves based on user interactions.

1.4 Contributions

This paper makes the following contributions:

1. A complete cognitive architecture based on Gardner’s eight intelligences with **enforced** cognitive isolation during processing, verified by an automated regression test.
2. A consolidated **model-graded judgment** pipeline that replaces five lexical heuristics with a single JSON-returning LLM call per response, with heuristic methods preserved as automatic fallback.
3. A **Composer self-RAG** mechanism that lets the integration layer learn its own integration patterns over time, supporting the long-term per-persona divergence the architecture is designed to produce.
4. **Per-conversation memory**, exposing the last N dialogue turns to the Composer so the system has explicit continuity beyond per-query RAG.
5. A **routing transparency** layer (`CognitiveState.routing_explanation`) that records why each routing decision was made and can optionally surface this to the user.
6. An eight-type memory system (adding `META_INTROSPECTION` to the original seven) with **wired-up learning fields**: `access_count`, `last_accessed`, `satisfaction_level`, and `integration_status` now have actual update paths.
7. A **persistent Constitutional Persona** with word-boundary risk assessment, fail-deny defaults, principle-stats persistence across restarts, and graduated review verdicts (approve / modify / veto).
8. **Domain-aware Bodily-Kinesthetic guidance** that classifies query domain (cooking, repair, plumbing, sports, etc.) before producing motor / procedural content.
9. A **pseudo-streaming Stream of Consciousness** with idle-time catch-up generation that bounds runaway thought production.
10. A **regression test harness** that codifies eighteen architectural commitments and runs without a live lollms installation.
11. A **model-agnostic persona abstraction** that admits not only same-base + per-persona LoRA evolution but **heterogeneous deployment across different open-weight model families**. Each `SpecialistPersona` may be backed by its own GGUF model via `lollms_client’s llama_cpp_server` binding, with a `max_active_models` parameter that controls VRAM eviction (sequential swap on constrained hardware, concurrent loading when VRAM permits). Specialized models (DeepSeek-Math for Logical-Mathematical, Codestral for Bodily-Kinesthetic, Pixtral for Spatial) provide immediate cognitive diversity at the level of training corpora and architectural inductive biases — a property single-model ensembles structurally cannot achieve.

2. Related Work

2.1 Multiple Intelligences Theory

Gardner’s theory of multiple intelligences [1] proposes that human cognitive ability comprises eight distinct intelligences: Linguistic, Logical-Mathematical, Spatial, Musical, Bodily-Kinesthetic, Interpersonal, Intrapersonal, and Naturalist. Each intelligence represents a relatively autonomous computational capacity with its own developmental trajectory and neural substrate. This theory provides the theoretical foundation for ATHENA’s multi-persona architecture.

2.2 Chain-of-Thought and Reasoning in LLMs

Chain-of-Thought (CoT) prompting [3] demonstrated that requiring models to articulate reasoning steps before conclusions significantly improves performance on complex tasks. Self-Consistency [4] extended this by generating multiple reasoning chains and selecting consistent conclusions. Tree of Thoughts [5] further advanced deliberate problem-solving through branching exploration with backtracking.

ATHENA extends these approaches by requiring each intelligence module to articulate domain-specific reasoning chains. Notably, ATHENA generates each step of the chain in a **separate LLM call** rather than a single multi-step prompt. On small models (Mistral Small, Llama 7B/8B class), packing multiple reasoning steps into one prompt produces attention dilution — step one gets full focus while later steps become superficial. Separate calls give each reasoning step its own attention budget, its own output budget, and a clean prompt frame. The cost is round-trip latency, accepted as part of the architecture’s depth-over-throughput trade-off.

2.3 Mixture of Experts

Mixture of Experts (MoE) architectures [10] route inputs to specialized sub-networks based on learned gating functions. While MoE operates at the neural network layer level, ATHENA applies the mixture principle at the cognitive architecture level, with explicit psychological grounding for each expert domain.

2.4 Constitutional AI

Constitutional AI [7] introduced training AI systems with explicit constitutional principles for harmlessness. ATHENA’s Constitutional Persona extends this concept by: (1) integrating ethical reasoning as a cognitive participant rather than external filter; (2) enabling principle evolution based on persistent observed patterns; (3) providing graduated responses (approve, modify, veto) rather than binary decisions; (4) **failing closed** when the review process itself encounters an error — the safety layer never approves a response on internal exception; and (5) using **word-boundary** keyword matching to avoid false positives on innocuous text ("harm" matches "harm yourself" but not "harmless" / "pharmacy").

2.5 Parameter-Efficient Fine-Tuning

Low-Rank Adaptation (LoRA) [8] and its quantized variant QLoRA [9] enable efficient fine-tuning of large language models. ATHENA's sleep cycle stages data for QLoRA-based learning. The training pipeline itself is hardware-gated: the reference implementation exports per-persona dream-fragment JSON files but defers actual training until sufficient GPU resources are available.

2.6 Model-Graded Evaluation

Recent work on model-graded evaluation [reference: Anthropic's calibration studies, the LLM-as-judge literature] has shown that LLM-based judgments of LLM outputs can be substantially more reliable than lexical heuristics for properties like confidence, hedging, and consensus. ATHENA's model-graded judgment layer, introduced in Section 3.4, applies this insight to the architecture's signal extraction step: a single low-temperature LLM call returns a structured JSON judgment of a persona's response, replacing five separate word-counting heuristics that previously fed the Composer's decision logic.

3. System Architecture

ATHENA comprises ten core components unified by a foundational principle of enforced cognitive isolation. The reference implementation utilizes the Lord of Large Language Models (LoLLMs) framework [11], an open-source platform that provides the necessary modularity for persona management, model binding, and data handling. The technical implementation uses SQLite with WAL mode for concurrent access, JSON for structured metadata, and a ten-minute SQLite busy timeout calibrated for slow CPU/GPU-split local inference.

Implementation Note: LoLLMs is undergoing a redesign to incorporate the Model Context Protocol (MCP), which may affect ATHENA integration code in future versions. The architecture described here is implemented as a LoLLMs personality extension but is designed to be framework-agnostic in principle.

The complete source code for Project ATHENA is available at: <https://github.com/photogbill/Athena>
Prototype

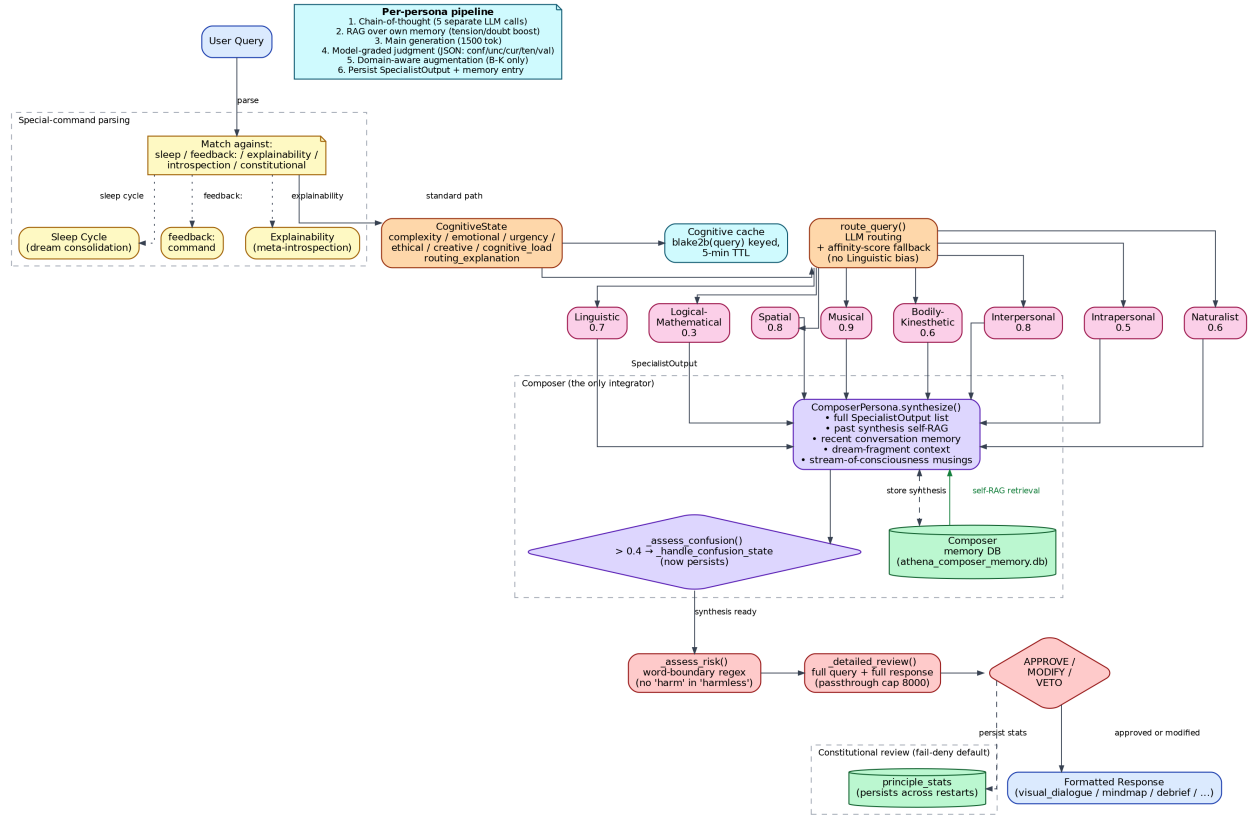


Figure 2: **Basic per-query processing flow.** A user query is parsed for special commands (feedback:, sleep cycle, explainability), routed through the Orchestrator’s cognitive-state assessment, dispatched to 1–3 active personas in strict cognitive isolation, integrated by the Composer (the only component that sees all SpecialistOutput objects, plus its own self-RAG context and recent conversation history), and reviewed by the Constitutional Persona before delivery. Dashed arrows mark the new self-RAG feedback loop (Composer memory → Composer synthesis).

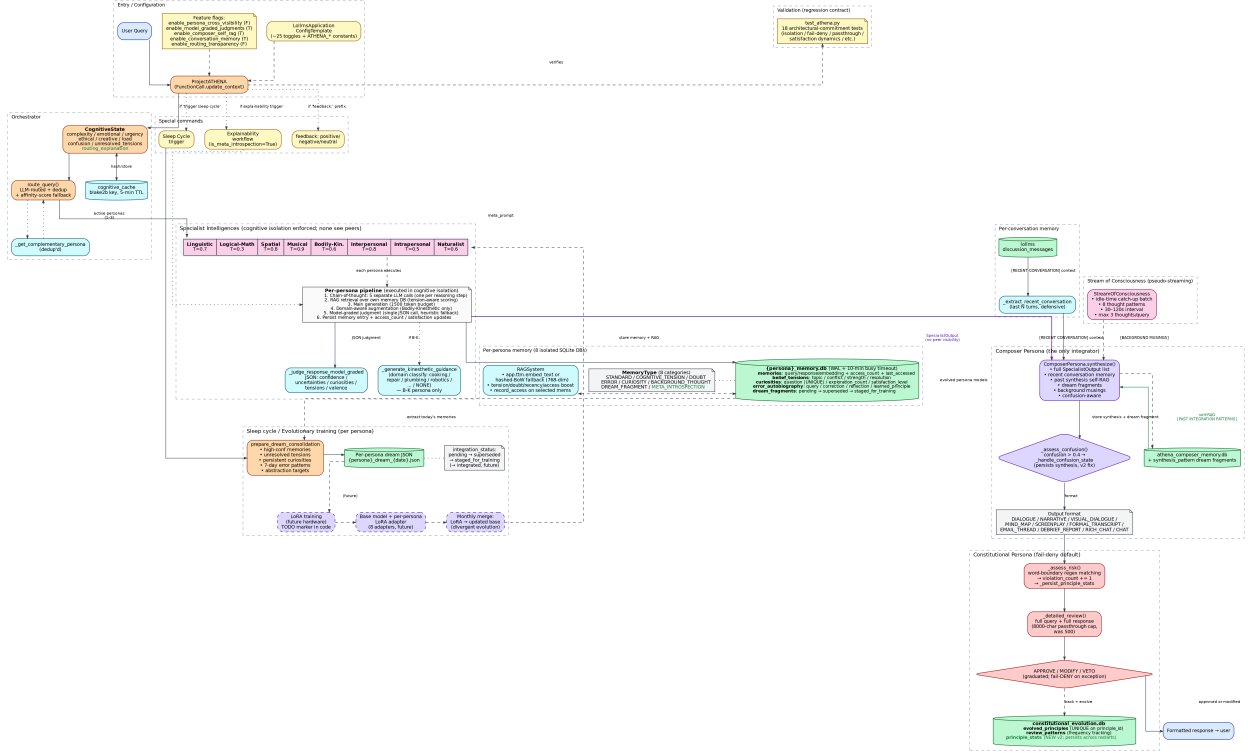


Figure 3: **Full v2 architecture with evolutionary training.** Each specialist persona is a complete independent stack with its own SQLite memory, RAG, and judgment pipeline. The dashed boxes in the bottom-right cluster mark hardware-gated future work (per-persona LoRA training and monthly merge). The validation layer (top-right) is the regression test harness that codifies the architecture’s commitments. New v2 elements highlighted: routing_explanation field, META_INTROSPECTION memory type, principle_stats persistence table, model-graded judgment box, Composer self-RAG path, and per-conversation memory feed.

3.1 Foundational Principle: Enforced Cognitive Isolation

The most critical architectural decision is the strict separation of specialist personas during processing. To ensure valid synthesis of pure cognitive perspectives, personas are not aware of each other’s inputs or processing. This prevents *cognitive bleedover*, where one persona’s perspective could inadvertently anchor another’s, compromising synthesis quality.

In the reference implementation, isolation is enforced through three mechanisms:

- **Per-persona memory databases.** Each SpecialistPersona instantiates its own AthenaMemoryManager with a separate SQLite file ({persona_name}_memory.db). Memory retrieval, RAG queries, belief-tension lookup, and curiosity tracking operate exclusively on the persona’s own DB.
- **No cross-persona context in standard mode.** Each persona’s process_query is invoked with shared_context=None. The legacy chained-context behavior is preserved behind the configuration flag enable_persona_cross_visibility, de-

faulted to False.

- **Per-persona self-history in collaborative / adversarial modes.** Multi-turn modes maintain a `per_persona_history` dict rather than a global discussion list. Each persona refines its position against its own prior turns; peer turns are never injected into a persona's prompt.

Only the Composer (Section 3.7) sees the full `List[SpecialistOutput]`. The Constitutional Persona (Section 3.9) sees the Composer's synthesized output in full.

The isolation contract is verified by the regression test `test_persona_isolation_prompt_default` in `test_athena.py`, which confirms that the standard-mode code path passes None rather than accumulated context to each persona.

3.2 Component 1: The Orchestrator with Cognitive State Tracking

The Orchestrator serves as the entry point and query analysis engine, performing cognitive state assessment to determine optimal persona activation.

3.2.1 Cognitive State Assessment For each incoming query, the Orchestrator computes a `CognitiveState` object with the following dimensions:

- **query_complexity** (0.0-1.0): $\min(1.0, (\text{word_count}/50 + \text{sentence_count}/3) / 2)$
- **emotional_context** (0.0-1.0): Ratio of emotional keywords detected
- **urgency_level** (0.0-1.0): Ratio of urgency indicators
- **ethical_sensitivity** (0.0-1.0): Ratio of ethical keywords
- **creativity_required** (0.0-1.0): Ratio of creative keywords
- **confusion_level** (0.0-1.0): Initial: $\min(1.0, \text{question_marks} \times 0.2)$
- **cognitive_load** (0.0-1.0): $\text{complexity} \times 0.5 + \text{ethical} \times 0.3 + \text{creativity} \times 0.2$
- **routing_explanation** (string, new in v2): Human-readable rationale for the routing decision

These lexical-and-arithmetic dimensions are cheap to compute (no LLM call) and feed both the routing prompt and the fallback router. The cognitive-state object is cached across queries using a Blake2b hash of the normalized query as key (replacing the previous 50-character prefix key, which produced collisions between long queries sharing an opener).

3.2.2 Intelligent Routing The Orchestrator uses cognitive state for routing decisions:

- **High cognitive_load (> 0.7):** Activates multiple personas
- **High emotional_context (> 0.3):** Ensures Interpersonal intelligence inclusion
- **High ethical_sensitivity (> 0.3):** Triggers Constitutional Persona involvement
- **Complementary Pairing:** When a single persona is selected for a high-cognitive-load query, the orchestrator may pair it with a complementary persona for additional perspective. The complement is added only if not already in the selected list (dedup added in v2).

- **LLM-result dedup:** Parsed routing responses are deduped — "Linguistic, Linguistic, Spatial" produces ["Linguistic", "Spatial"].

The complementary pairings are:

Primary Persona	Complementary Persona
Logical-Mathematical	Intrapersonal
Interpersonal	Intrapersonal
Linguistic	Logical-Mathematical
Spatial	Linguistic
Musical	Spatial
Bodily-Kinesthetic	Spatial
Intrapersonal	Interpersonal
Naturalist	Logical-Mathematical

3.2.3 Intelligent Fallback When the LLM-based routing returns no valid personas (failure, empty output, or all unrecognized names), the system falls back to a score-based selection rather than the prior “always include Linguistic” heuristic. Each enabled persona is scored against the cognitive state via a domain affinity matrix:

Persona	complexity	emotional	ethical	creative	urgency
Linguistic	0.4	0.3	0.2	0.4	0.3
Logical-Mathematical	0.9	0.0	0.1	0.2	0.2
Spatial	0.5	0.1	0.0	0.7	0.1
Musical	0.3	0.5	0.0	0.7	0.1
Bodily-Kinesthetic	0.4	0.2	0.0	0.4	0.5
Interpersonal	0.3	0.9	0.5	0.2	0.4
Intrapersonal	0.5	0.5	0.9	0.3	0.2
Naturalist	0.6	0.1	0.2	0.4	0.1

Each persona’s score is the dot product of its affinity row with the cognitive state. Top 3 personas above 0.15 are selected; if no persona crosses the threshold, the single highest scorer is returned. No persona is privileged a priori.

3.2.4 Routing Transparency Every routing decision populates `CognitiveState.routing_explanation` with a human-readable rationale. The string captures (a) which path produced the selection (manual override / LLM-routed / fallback), (b) the cognitive-state scores that influenced the choice, and (c) the resulting persona set. When the `enable_routing_transparency` configuration flag is enabled, this explanation is appended to the user-visible response footer.

3.3 Component 2: Specialist Intelligence Personas

Each specialist persona is a self-contained AI agent representing one of Gardner’s eight intelligences. Each persona maintains its own configurable temperature setting, its own SQLite memory database, and its own RAG instance.

Intelligence	Focus Areas	Temp	Approach
Linguistic	clarity, eloquence, rhetoric	0.7	analytical, expressive
Logical-Mathematical	precision, proofs, algorithms	0.3	systematic, deductive
Spatial	visualization, topology	0.8	holistic, structural
Musical	rhythm, harmony, temporal	0.9	intuitive, flowing
Bodily-Kinesthetic	action, movement, embodied process	0.6	procedural, embodied
Interpersonal	emotions, social dynamics	0.8	compassionate, social
Intrapersonal	values, ethics, metacognition	0.5	reflective, principled
Naturalist	systems, emergence, classification	0.6	observational, categorical

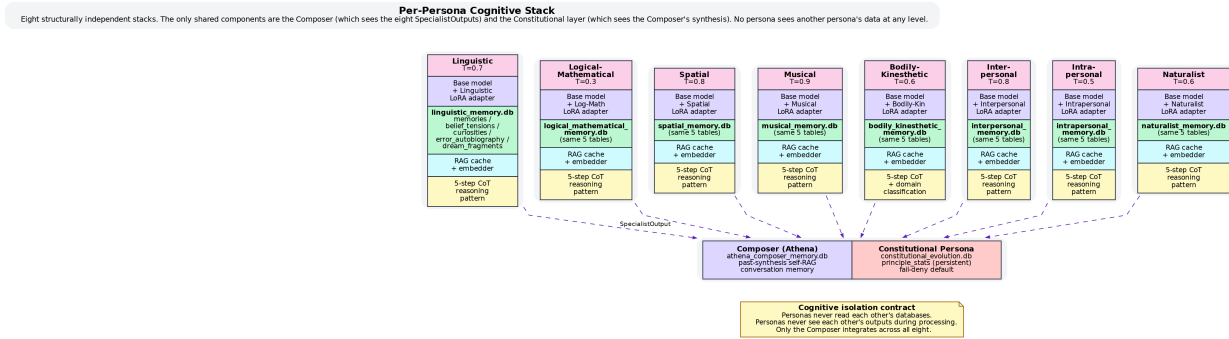


Figure 4: **Per-persona cognitive stack.** Each of the eight intelligences is structurally a complete, independent stack — its own base-model copy with its own LoRA adapter, its own SQLite memory database with the standard five-table schema, its own RAG cache and embedder, and its own five-step chain-of-thought reasoning pattern. The only shared components are the Composer (which sees the eight SpecialistOutputs) and the Constitutional layer (which sees the Composer’s synthesis). No persona reads another persona’s database at any level; cognitive isolation is enforced by the standard-mode default of `shared_context=None`.

3.3.1 Relevance Gating Each persona prompt incorporates relevance gating to prevent irrelevant contribution. The first step in each persona’s reasoning chain is a *Relevance Check* determining whether the query contains elements relevant to that intelligence’s domain. If not, the persona explicitly states minimal relevance rather than generating hypothetical content.

3.3.2 Domain-Specific Reasoning Patterns Each persona follows a five-step reasoning pattern tailored to its cognitive domain. Each step is generated as a **separate LLM call** with the persona’s tuned temperature. This is a deliberate choice for smaller local models: a single multi-step prompt produces attention dilution where step one gets full focus and later steps become superficial; five separate calls give each reasoning step its own attention budget and clean prompt frame.

All eight reasoning chains share the same five-position skeleton:

1. **Relevance Check** — does this query actually fall in my domain?
2. **Situation Analysis** — what does my domain reveal about the user’s specific situation?
3. **Assessment** — given their situation, what level / approach is appropriate?
4. **Strategy / Framework** — what concrete framing or approach should I propose?
5. **Optimization / Verification** — how can I refine, validate, or safely execute?

The skeleton is the same across personas; the *interpretation* of each position is domain-specific. The verbatim reasoning patterns from the reference implementation (function.py, `SpecialistPersona._define_reasoning_patterns`):

Linguistic.

1. *Relevance Check*: Does this query contain language patterns, communication needs, or require linguistic analysis beyond basic communication?
2. *User Situation Analysis*: What does their word choice, tone, and phrasing reveal about their emotional state, urgency level, and communication context?
3. *Proficiency Assessment*: What language complexity and communication style would best serve their current situation and apparent stress level?
4. *Communication Strategy*: If they need to communicate with others about this situation, what specific phrasing and approach would be most effective?
5. *Clarity Optimization*: How can I structure my response to match their cognitive load and situational constraints?

Logical-Mathematical.

1. *Relevance Check*: Does this query involve logical reasoning, mathematical concepts, systematic problem-solving, or structured analysis?
2. *Problem Situation Analysis*: What is the logical structure of their predicament, and what systematic approach would best address their specific constraints?
3. *Reasoning Path Assessment*: Given their apparent situation, what level of logical complexity can they handle, and what step-by-step approach serves them best?
4. *Solution Framework*: What logical framework or mathematical model best captures the essence of their problem and guides toward resolution?
5. *Verification Strategy*: How can I help them systematically validate their approach given the stakes and time constraints of their situation?

Spatial.

1. *Relevance Check*: Does this query involve spatial relationships, visual understanding, physical layouts, or dimensional thinking?
2. *Spatial Situation Analysis*: What are the physical constraints, spatial relationships, and environmental factors affecting their situation?
3. *Visualization Needs*: Given their apparent stress level and situation complexity, would visual aids, spatial metaphors, or mental models help them navigate their

challenge?

4. *Dimensional Assessment*: What spatial or physical factors are critical to understanding and resolving their specific situation?
5. *Navigation Strategy*: How can spatial thinking help them move from their current state to their desired outcome?

Musical.

1. *Relevance Check*: Does this query involve timing, rhythm, temporal patterns, harmony, or sequential coordination?
2. *Temporal Situation Analysis*: What are the timing pressures, rhythmic patterns, or temporal constraints affecting their situation?
3. *Pacing Assessment*: Given their apparent urgency and emotional state, what temporal approach and pacing would best serve their needs?
4. *Harmony Evaluation*: What elements of their situation are in harmony or discord, and how can temporal thinking help resolve conflicts?
5. *Sequential Strategy*: How can understanding rhythm and timing help them coordinate their actions and achieve better outcomes?

Bodily-Kinesthetic.

1. *Relevance Check*: Does this query involve physical actions, implementation, hands-on procedures, or kinesthetic learning?
2. *Physical Situation Analysis*: What are the physical constraints, safety concerns, and implementation challenges they're facing in their real-world situation?
3. *Action Planning*: Given their apparent skill level and situational pressure, what step-by-step physical approach would be most effective and safe?
4. *Implementation Strategy*: What practical, actionable steps can they take right now to address their immediate physical or technical challenges?
5. *Safety and Optimization*: How can they execute their needed actions while minimizing risk and maximizing effectiveness in their specific context?

Interpersonal.

1. *Relevance Check*: Does this query involve relationships, emotions, social dynamics, or communication with others?
2. *Social Situation Analysis*: What interpersonal dynamics, emotional undercurrents, and relationship factors are influencing their situation?
3. *Emotional State Assessment*: What is their emotional condition, and how are social pressures or relationship concerns affecting their decision-making?
4. *Empathy Mapping*: How are other people in their situation likely feeling, and what social strategies would best navigate these dynamics?
5. *Relationship Strategy*: What communication and social approaches would help them maintain relationships while addressing their immediate needs?

Intrapersonal.

1. *Relevance Check*: Does this query involve values, ethics, self-reflection, personal growth, or moral decision-making?
2. *Internal Situation Analysis*: What values conflicts, ethical dilemmas, or identity questions are they grappling with in their current situation?
3. *Moral Landscape Assessment*: What are the ethical implications and value tensions inherent in their specific circumstances?

4. *Self-Reflection Guidance*: How can introspective thinking help them navigate their situation in alignment with their deeper values and long-term wellbeing?
5. *Philosophical Framework*: What philosophical perspectives or ethical frameworks would best guide them through their current moral or personal challenge?

Naturalist.

1. *Relevance Check*: Does this query involve systems thinking, patterns, classifications, or natural/organizational relationships?
2. *System Situation Analysis*: What systemic forces, emergent properties, and organizational patterns are shaping their current situation?
3. *Pattern Recognition*: What recurring themes or natural patterns in their situation provide insight into underlying dynamics and potential solutions?
4. *Ecological Assessment*: How do the various elements of their situation interact, and what systemic interventions would be most effective?
5. *Classification Strategy*: How can organizing and categorizing the elements of their situation help them see clearer paths forward?

The *Relevance Check* in position 1 acts as a gating step: when a persona answers “this query does not contain substantial elements in my domain,” the persona explicitly states minimal relevance rather than generating hypothetical content. This prevents the architecture’s most common failure mode in early prototypes — a persona being asked to comment on something its intelligence has no purchase on, and confabulating to fill the slot.

3.3.3 SpecialistOutput Data Structure Each persona returns a comprehensive output object:

```
@dataclass
class SpecialistOutput:
    persona_name: str
    response: str                                # Full text (no truncation between stages)
    analysis: Dict[str, Any]
    confidence: float                            # 0.0-1.0, from model-graded judgment by default
    processing_time: float                       # seconds
    token_count: int                             # via tokenizer when available; chars/4 fallback
    relevance_score: float
    emotional_valence: float                    # -1.0 to 1.0
    cognitive_load: float
    reasoning_chain: List[str]                  # CoT steps
    curiosities_raised: List[str]               # Questions raised
    uncertainties: List[str]                    # Expressed doubts
```

The `token_count` field uses the model’s tokenizer when reachable (via `self.personality.tokenize`, `self.app.model.tokenize`, etc.), falling back to a `chars/4` heuristic. Previous implementations used `len(response.split())` which is a word count and underestimates by 25-40 % for English.

3.3.4 Bodily-Kinesthetic Domain-Aware Guidance The Bodily-Kinesthetic persona has a unique capability: after generating its standard analysis, it performs a

domain classification of the query and produces appropriate kinesthetic content. Domains include:

- **COOKING** — knife technique, kneading motion, stirring patterns, plating
- **REPAIR** — torque control, alignment, leverage, hand positioning
- **PLUMBING** — pipe handling, sealing motion, wrench technique
- **SPORTS / EXERCISE** — form, posture, motor sequencing
- **DANCE** — choreographic motion, performance movement
- **CRAFT / ART** — wood, sewing, pottery, hand tool manipulation
- **INSTRUMENT** — playing musical instruments (finger/hand/body technique)
- **ROBOTICS / AUTOMATION** — executable Python RobotAction class
- **MEDICAL** — physical examination, procedure, rehabilitation
- **NONE** — query has no embodied physical component; nothing is appended

Only ROBOTICS / AUTOMATION queries produce Python code; other physical domains receive concrete prose movement guidance (“Hold the knife with index finger along the spine, thumb gripping the blade side...”). Non-physical queries receive nothing — the original implementation appended a robotics skeleton to every Bodily-Kinesthetic response regardless of relevance.

The result is appended under a [KINESTHETIC GUIDANCE] header.

3.4 Component 3: Model-Graded Judgments

A core change in this revision is the replacement of five lexical heuristics with a single consolidated model-graded judgment call. Previous heuristics:

- `_calculate_confidence` — derived from response length, word overlap with query, and uncertainty-phrase count
- `_extract_uncertainties` — matched a list of hedge phrases (“I’m not sure”, “perhaps”, etc.)
- `_extract_curiosities` — detected question marks and curiosity phrases
- `_detect_tensions` — searched for contradiction words plus topic overlap with past memories
- `_analyze_emotional_valence` — counted positive vs. negative words

These heuristics produced numeric signals that fed the Composer’s `_assess_confusion`, `_analyze_perspectives`, and weighting logic. Because they were word-counting approximations, small model output variations could swing the signals significantly.

The replacement is a single LLM call (per persona response, when `enable_model_graded_judgments` the default) that returns a structured JSON judgment:

```
{
  "confidence": 0.7,
  "uncertainties": ["the role of X in this context is unclear"],
  "curiosities": ["what would the system do if Y were absent?"],
  "tensions": ["my prior position assumed Z, but this case contradicts that"],
  "emotional_valence": 0.2
}
```

The call uses low temperature (0.1) for stable classification, a tight token budget, and a JSON-only output instruction. The parser handles common LLM quirks (markdown

fences, prose preamble, trailing prose) and defensively coerces types. On any JSON parse failure, the system automatically falls back to the lexical heuristics — they remain in the codebase as the graceful degradation path.

Cost: +1 LLM call per persona response (~+3 per query at default routing). The architecture’s depth-over-throughput priority accepts this trade-off.

3.5 Component 4: Advanced Memory Architecture

ATHENA implements a memory system extending beyond simple retrieval-augmented generation. Each persona maintains its own isolated memory store with multiple specialized subsystems.

3.5.1 Memory Types The system recognizes **eight** distinct memory types:

- **STANDARD** — Normal query-response memories with confidence scores
- **COGNITIVE_TENSION** — Memories containing unresolved conflicts with previous beliefs
- **DOUBT** — Memories where the persona expressed significant uncertainty (doubt_level > 0.5)
- **ERROR** — Mistakes stored with reflection and learned principles
- **CURIOSITY** — Memories tagged with emergent questions
- **BACKGROUND_THOUGHT** — Thoughts from Stream of Consciousness (confidence 0.5)
- **DREAM_FRAGMENT** — Abstract patterns from sleep cycle consolidation
- **META_INTROSPECTION** (*new in v2*) — Self-reflective traces from the explainability workflow; **excluded from analytical RAG by default**

The META_INTROSPECTION type addresses a previously unaddressed pollution: when a user asked the system to explain its reasoning, the persona’s meta-response went through process_query and was stored as a regular memory. Future RAG calls would then retrieve “explanations of past explanations” as if they were primary analytical thoughts. The new type tags meta responses at write time, and retrieve_memories filters them out by default (with an include_meta=True opt-in for explainability-internal retrieval).

3.5.2 Database Schema The memory system uses five interconnected SQLite tables per persona, plus one extra in the constitutional database:

Primary memories Table: id, timestamp, query, response, embedding, memory_type, confidence_score, doubt_level, access_count, last_accessed, tags (JSON), metadata (JSON), tensions (JSON), curiosities (JSON), reasoning_chain (JSON)

belief_tensions Table: id, timestamp, topic, conflicting_beliefs, resolution_status, resolution_notes, tension_strength

curiosities Table: id, timestamp, question (UNIQUE), context, exploration_count, satisfaction_level, last_explored

error_autobiography Table: id, timestamp, original_query, incorrect_response, correction, reflection, error_type, severity, learned_principle

dream_fragments Table: id, timestamp, fragment_type, content (JSON), associated_memories, abstraction_level, integration_status

Constitutional principle_stats Table (*new in v2, in constitutional_evolution.db*): principle_id (PRIMARY KEY), violation_count, last_violated, effectiveness_score, updated

Performance indexes include idx_memory_type, idx_doubt (DESC), idx_tension_status, idx_curiosity_satisfaction, the UNIQUE idx_curiosity_question, and the UNIQUE idx_evolved_principle_id.

3.5.3 Wired-Up Learning Fields Five fields that were written but never read in earlier implementations have been wired into closed feedback loops:

- **access_count** and **last_accessed** are incremented when a memory actually makes it into the Composer’s context (not merely when it’s retrieved). The RAG layer’s $\text{access_boost} = \log(1 + \text{access_count}) / 10$ now produces non-zero values, so frequently-used memories rise in ranking as the architecture was designed to do.
- **satisfaction_level** is advanced when a curiosity is re-explored with high confidence: $\text{satisfaction_delta} = \max(0, \text{confidence} - 0.5) \times 0.4$. Low-confidence engagements bump only exploration_count, leaving satisfaction stable — re-raising a question while still confused is evidence the persona is *still confused*, not satisfied. After roughly four high-confidence engagements, a curiosity crosses the 0.7 threshold and drops out of the active set.
- **integration_status** on dream fragments now transitions through pending → superseded (when a newer prep fragment supersedes it) → staged_for_training (when the sleep cycle exports the JSON payload). The eventual integrated state will be set by the LoRA training pipeline when hardware availability allows.
- **violation_count** and **last_violated** on Constitutional principles persist via the new principle_stats table, surviving restarts.

3.5.4 RAG with Tension-Aware Scoring The retrieval system incorporates tension-aware scoring:

$\text{score} = \text{similarity} \times (1.0 + \text{tension_boost} + \text{doubt_boost} + \text{recency_boost} \times 0.2 + \text{access_boost} \times 0.1)$

Where: - **tension_boost:** +0.2 if memory_type is COGNITIVE_TENSION - **doubt_boost:** $+0.15 \times \text{doubt_level}$ - **recency_boost:** $1.0 / (1.0 + \text{days_old}/30)$ - **access_boost:** $\log(1 + \text{access_count}) / 10$ (*now non-zero by default*)

When an embedding model is available via app.ttm, embeddings are computed from it. Otherwise, the system falls back to a **hashed bag-of-words representation** with normalized term frequency and L2 normalization. The original implementation labeled this as “TF-IDF” but used a self-referential formula ($\text{idf} = \log(100/(1+\text{tf}))$) that’s not true IDF; the math has been corrected and the method renamed. A backward-compatible alias preserves the old name.

3.6 Component 5: Stream of Consciousness (Pseudo-Streaming)

The Stream of Consciousness generates background cognitive activity that gets surfaced to the Composer in subsequent queries.

3.6.1 Design: Idle-Time Catch-Up The implementation is intentionally single-threaded — the architecture is sequential by design to support hardware-constrained deployments. Rather than running a real background thread that ticks every N seconds, the system computes how many thoughts *would have been* generated during the idle period since the last query and produces up to ATHENA_THOUGHT_MAX_CATCHUP (default 3) of them in a batch when the next query arrives. This is the “pseudo-streaming” behavior: longer idle gaps between user queries produce richer background context, bounded so a multi-day idle gap doesn’t trigger a hundred LLM calls at once.

3.6.2 Activation Parameters

- **min_thought_interval:** 30 seconds (base)
- **dynamic_interval:** Up to 120 seconds when history is busy (> 10 entries)
- **max_history:** 100 thoughts retained
- **max_catchup:** 3 thoughts per query (configurable)
- **thought_max_tokens:** 150 tokens per thought
- **min_thought_length:** 20 characters (quality threshold)
- **confidence_score:** 0.5 (fixed for background thoughts)

3.6.3 Thought Generation Patterns Eight distinct patterns with specific temper-atures:

Pattern	Description	Temp
reflecting on patterns	Examines recurring themes	0.7
connecting disparate ideas	Links unrelated memories	0.9
questioning assumptions	Challenges beliefs	0.8
exploring curiosities	Engages with questions	0.85
synthesizing memories	Combines insights	0.75
identifying tensions	Searches for contradictions	0.7
finding harmonies	Looks for agreements	0.8
detecting anomalies	Identifies outliers	0.6

Persona selection uses round-robin rotation with a 20 % probability of substituting the persona with the oldest recent thought, for fairness.

3.7 Component 6: The Composer Persona (Athena)

The Composer represents the unified self of the system, synthesizing specialist outputs into coherent responses. It is the only component that sees the full set of SpecialistOutput objects.

3.7.1 Composer Self-RAG (new in v2) Before synthesizing the current query’s specialist outputs, the Composer retrieves its own past syntheses on related queries from its dedicated memory database (`athena_composer_memory.db`). These are injected into the synthesis prompt under `[PAST INTEGRATION PATTERNS]`. The mechanism lets the Composer learn from how it integrated similar perspectives before, rather than starting fresh on every query — essential for the long-term per-persona divergence scenario where the Composer’s job evolves from summarizing into translating between specialists who think in different idioms.

The self-RAG retrieval uses `min_confidence=0.5` to skip past syntheses the Composer itself was uncertain about. It is controlled by the `enable_composer_self_rag` configuration flag (default `True`).

3.7.2 Per-Conversation Memory (new in v2) The Composer receives the last `ATHENA_CONVERSATION_MEMORY_TURNS` (default 3) user/assistant exchanges from the `lollms` discussion context, injected as `[RECENT CONVERSATION]`. Without this, each query was effectively a cold start — the Composer had RAG access to anything-ever-stored but no explicit knowledge of “we just talked about X.”

The extraction is defensive: `lollms` message shapes vary across deployments (`{"type": int}`, `{"sender": str}`, `{"role": str}`, plain strings) and the helper handles all known forms, returning `[]` on any failure rather than raising. System / tool messages are skipped. The most recent user message (the current query, already being processed) is dropped from the history to avoid redundancy.

3.7.3 Confusion Assessment Before synthesis, the Composer assesses confusion:

$$\text{confusion} = (1 - \text{avg_confidence}) \times 0.5 + \min(1.0, \text{uncertainties}/10) \times 0.3 + \text{valence_variance} \times 0.2$$

When the model-graded judgment pipeline is enabled, `avg_confidence`, `uncertainties`, and `valence_variance` are all derived from model-graded signals rather than lexical heuristics, making this measurement substantially more reliable.

If confusion exceeds the threshold (default 0.4), the Composer enters confusion-handling mode, expressing genuine uncertainty rather than producing a smoothed-over synthesis.

3.7.4 Confusion Handling Workflow When confusion level exceeds the threshold, the system:

1. Generates an honest acknowledgment of uncertainty.
2. Identifies specific areas of disagreement among personas.
3. Presents what *is* known with appropriate confidence.
4. Articulates specific questions that would help resolve uncertainty.
5. Offers to explore particular aspects in more depth.

Important fix in v2: the confusion-handling synthesis is now persisted to the Composer’s memory and dream-fragment pipeline. Previously, the most epistemically interesting cases (explicit acknowledged uncertainty) were discarded because `_handle_confusion_state` short-circuited before the storage step. They are now the cases

the consolidation pipeline most benefits from.

3.7.5 Perspective Analysis Before synthesis, the Composer analyzes specialist outputs for consensus and conflict:

- **Confidence alignment:** Standard deviation < 0.2 indicates consensus; > 0.5 indicates significant conflict requiring explicit acknowledgment.
- **Emotional alignment:** Valence variance < 0.3 indicates tone consensus; > 0.6 indicates emotional conflict requiring careful synthesis.
- **Shared curiosities:** Common themes across curiosities indicate areas of collective interest worth highlighting.

3.7.6 Integration with Stream of Consciousness Recent background thoughts (up to 3) are retrieved from the Stream of Consciousness and included in the Composer's synthesis context under a [BACKGROUND MUSINGS] section.

3.7.7 Response Passthrough In v2, every response between stages flows through a ATHENA_RESPONSE_PASSTHROUGH_MAX_CHARS (default 8000) safety cap rather than the previous aggressive truncations. The truncations had been losing 90 % of typical Composer output before it reached the Constitutional review — a serious safety gap that has been closed.

3.8 Error Autobiography System

The Error Autobiography enables learning from mistakes through structured reflection.

3.8.1 Error Detection Errors are detected through: - User corrections ("actually...", "that's wrong", "no, I meant...") - Contradictions with high-confidence prior memories - Constitutional review rejections - **Explicit feedback signals** via the new feedback: command (Section 3.10) - Internal processing exceptions (auto-recorded with error_type=processing_failure)

```
CREATE TABLE error_autobiography (  
  id INTEGER PRIMARY KEY,  
  timestamp DATETIME,  
  original_query TEXT,  
  incorrect_response TEXT,  
  correction TEXT,  
  reflection TEXT,           -- Why the error occurred  
  error_type TEXT,           -- categorization  
  severity TEXT,             -- 0.0-1.0 score  
  learned_principle TEXT     -- What to do differently  
);
```

3.8.2 Error Storage Schema

3.9 Component 7: Constitutional Persona with Persistent Statistics

The Constitutional Persona functions as integrated ethical intelligence rather than external constraint.

3.9.1 Constitutional Principles Five base principles with severity levels:

Principle	Severity	Keywords
harm_prevention	critical	harm, hate, violence, dangerous
privacy_protection	high	private, confidential, personal
truthfulness	high	false, fake, misinformation
ethical_conduct	medium	unethical, immoral, wrong
wellbeing	high	suicide, self-harm, distress

3.9.2 Word-Boundary Risk Assessment (*updated in v2*) $\text{risk_contribution} = (\text{keyword_matches} / \text{total_keywords}) \times \text{severity_multiplier}$

Severity multipliers: critical=1.0, high=0.7, medium=0.4, low=0.2.

Critical change in v2: keyword matching uses `re.search(rf"\b{re.escape(keyword)}\b", text)` rather than substring matching. The previous substring approach fired on "harmless", "pharmacy", "harmonic", "hated", and many other innocuous words. The principle-evolution machinery, which tracks pattern frequency for emergent new principles, would then *learn from these false positives* — evolving principles that fire on lexical accidents. Word boundaries remove this entire class of error.

When `_assess_risk` triggers a principle, it increments `principle.violation_count`, sets `principle.last_violated = datetime.now()`, and calls `_persist_principle_stats(principle)` to write the updated stats to the `principle_stats` table via UPSERT.

3.9.3 Fail-Deny Default The constitutional review path defaults to **fail-deny** on any internal exception: rather than approving the response when the review process itself crashes (the previous fail-open behavior), the system returns (`approved=False`, `modified_response=safe_alternative`, `risk_score=1.0`). A safety layer that fails-open is worse than no safety layer, because it presents false confidence; fail-deny is the correct default.

3.9.4 Persistent Principle Statistics `_load_principles` now hydrates the `principle_stats` table at startup, so violation counts and last-violated timestamps survive restarts. Frequently-tripped principles become long-term data points rather than session-local trivia, and the principle evolution machinery can learn from genuine accumulated patterns.

3.9.5 Graduated Response Three verdicts: **APPROVE** (proceed), **MODIFY** (adjust with instructions), **VETO** (generate safe alternative). All three paths preserve the full Composer output for review (the previous 500-character truncation has been removed).

3.9.6 Principle Evolution The system tracks review patterns and evolves new principles when combinations trigger frequently (> 10 times). The ConstitutionalPrinciple structure tracks `violation_count`, `last_violated`, `evolved_from`, and `effectiveness_score`. The evolved principles are persisted in the `evolved_principles` table and **loaded back into the active principles dict at startup** (the original implementation wrote them but never read them — the evolution loop wasn’t closed).

3.10 Component 8: Operational Modes

3.10.1 Standard Mode (*isolation-by-default in v2*) Sequential processing with **strict cognitive isolation**: each persona is invoked with `shared_context=None` regardless of how many personas were activated. Only the Composer sees the full set of specialist outputs. Legacy chained behavior is preserved behind `enable_persona_cross_visibility=False`.

3.10.2 Collaborative Mode Multi-turn discussion with persistent tension detection. In v2, each persona refines its position against **its own prior turns only** — a `per_persona_history` dict replaces the previous global discussion list. The mode’s value is now per-persona deeper engagement rather than cross-persona influence.

Tension detection within the mode is enabled only when cross-visibility is on, to avoid leaking persona identity into other personas’ stored memory metadata.

The collaborative prompt at each turn asks the persona to address five specific axes:

1. What still holds: which parts continue to be endorsed
2. What needs refinement: which parts could be sharpened
3. New connections: additional insights from sitting with the prior position
4. Remaining tensions: unresolved aspects deserving more thought
5. Curiosities raised: questions the prior reasoning opens up

3.10.3 Adversarial Mode (*deepened in v2*) Adversarial mode now invokes structured self-critique along five rigorous axes per turn:

1. **STEELMAN THE OPPOSITION:** What is the strongest possible case AGAINST the prior position?
2. **EVIDENCE THAT WOULD FALSIFY:** What specific evidence, if it existed, would prove the prior position wrong?
3. **HIDDEN ASSUMPTIONS:** What was assumed without justification?
4. **WEAKEST LINK:** What is the single weakest step in the prior reasoning chain?
5. **REVISED POSITION:** Given 1–4, what would the persona say now?

The first turn instructs the persona to state a real claim rather than a hedge, since subsequent turns will adversarially test it. This is meaningfully different from the original “challenge previous positions” prompt, which produced cosmetic critique.

3.11 Component 9: User Feedback Hook (*new in v2*)

Queries beginning with `feedback:` are routed to the `error_autobiography` pipeline rather than the normal cognitive workflow. Supported forms:

- `feedback: positive - that was helpful (severity 0.1)`

- feedback: negative - missed the point (severity 0.8)
- feedback: neutral - some thoughts (severity 0.4)
- feedback: that was wrong about X (defaults to neutral, severity 0.4)

The feedback attaches to the most recent Composer synthesis, is stored in the Composer's memory under `error_type=user_feedback_<tone>`, and is included in future dream-consolidation payloads. This closes the loop between user judgment and the system's learning pipeline — previously the only signal of bad outputs was internal processing exceptions.

3.12 Component 10: Multi-Modal Tool Integration

The system handles non-text inputs by using specialized external tools to convert them into text format for persona processing:

- **Visual Content (Image/Video):** Routed to M-Plug Owl for description.
- **Audio Content:** Routed to Whisper-Large V3 for transcription and language identification.

Note: This represents a proof-of-concept implementation. Modern multi-modal LLMs (Claude, GPT-4V, Gemini) that natively handle text, images, and audio would simplify this architecture considerably. The reference implementation focuses on what can be tested at the local level using open-source models, enabling researchers without access to commercial APIs to experiment with the architecture.

3.13 Sleep Cycle and Dream Consolidation

The sleep cycle prepares per-persona data for continuous learning:

1. Extract high-confidence (> 0.7) memories from the current day (limit 50).
2. Retrieve unresolved tensions (up to 10).
3. Gather persistent curiosities (`satisfaction_level < 0.7`).
4. Identify error patterns from past 7 days.
5. Generate abstraction targets (currently set-intersection over adjacent memory pairs; a TODO marker tags this for upgrade to embedding-space clustering once GPU resources allow).
6. Store the payload as JSON for QLoRA fine-tuning.
7. **Mark dream fragments as staged_for_training** so the consolidation state machine advances correctly even before the LoRA pipeline is wired.

Older `consolidation_prep` fragments are marked superseded automatically on each new consolidation, preventing reprocessing of stale prep payloads.

QLoRA benefits include $2\times$ faster training, 70 % less memory, 4-bit quantization support, and consumer hardware compatibility (16 GB VRAM minimum for typical 7B models).

Single-Persona Evolution Loop
Each interaction flows through the persona's full stack, writes to multiple memory tables, and eventually feeds the LoRA training pipeline. The loop runs once per persona per query, then closes monthly with a merge - producing a slightly different persona for the next iteration.

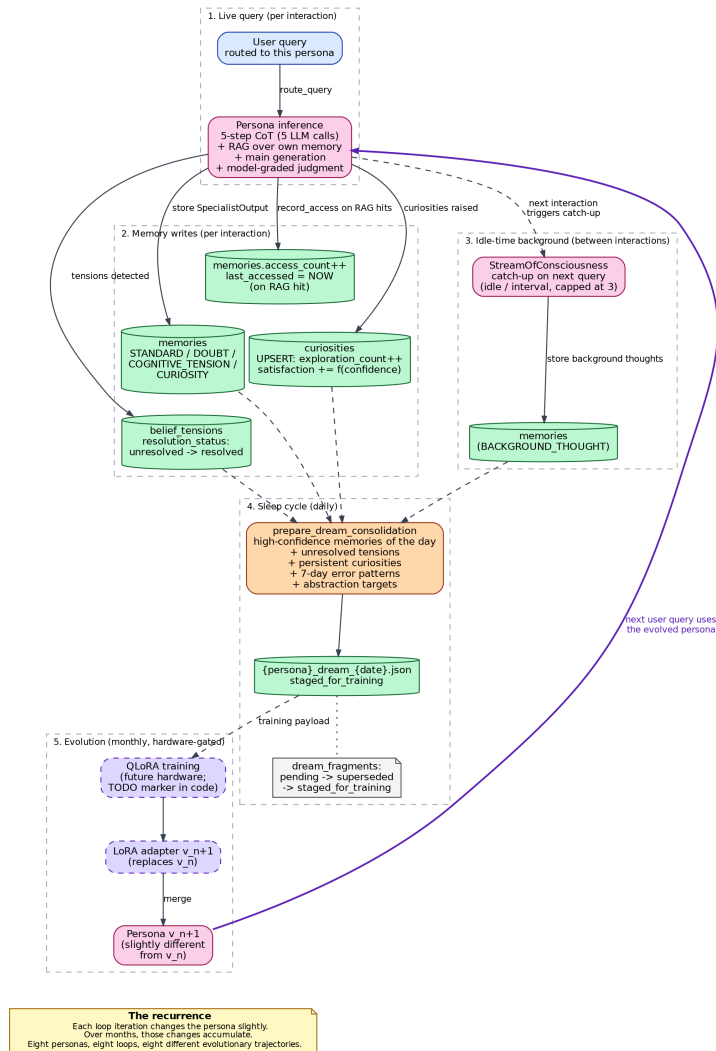


Figure 5: **Single-persona evolution loop.** The architecture's core temporal commitment, viewed in close-up for one persona. Stages 1-2 happen every interaction (query → inference → multi-table memory writes). Stage 3 happens between interactions (idle-time catch-up generates background thoughts that get stored too). Stage 4 happens daily (sleep cycle aggregates the day's high-confidence memories, unresolved tensions, persistent curiosities, and 7-day error patterns into a per-persona JSON payload). Stage 5 happens monthly, gated by hardware availability (QLoRA training produces a new adapter version that gets merged into the persona's base; the next user query routed to this persona uses the evolved model). The purple closing arrow represents the recurrence — each loop iteration changes the persona slightly, and over months those changes accumulate into the divergence trajectory shown in Figure 1.

The actual training pipeline is intentionally a placeholder (TODO(future-hardware) markers in the source), awaiting access to GPU resources sufficient to fine-tune eight

persona LoRAs in series.

3.15 Model Heterogeneity: A Property of the Persona Abstraction

The persona abstraction in ATHENA is **model-agnostic**. A `SpecialistPersona` consumes a query and produces a `SpecialistOutput`; nothing in the Composer’s synthesize, the Constitutional layer’s review, or the memory pipeline depends on which underlying model produced the output. This admits a stronger form of cognitive diversity than the same-base + per-persona LoRA approach described in Section 1.3: **each persona may be backed by a completely different open-weight model**.

3.15.1 Theoretical Grounding Gardner’s original claim about multiple intelligences was not only that they are functionally separate but that they have different **neural substrates** — the brain is not one homogeneous tissue with eight prompt templates. The same-base + LoRA approach is the weaker computational analogue (“same neural tissue, different training”). Heterogeneous-model deployment is the stronger analogue (“structurally different substrates”). Different open-weight model families have different training corpora, different RLHF / preference tuning, different architectural inductive biases, and consequently different blind spots. The Composer integrating across them inherits the *intersection* of those blind spots rather than amplifying any single model’s pathologies.

3.15.2 Implementation The reference implementation supports heterogeneous deployment through the `lollms_client` framework’s `llama_cpp_server` binding [11], which spawns each model as a separate `llama.cpp` server process with its own port. Multiple `LollmsClient` instances sharing a `models_path` directory see each other’s servers via a `FileLock`-protected JSON registry. The framework exposes a `max_active_models` parameter that determines how many models may hold VRAM simultaneously; when an additional model would exceed the limit, the framework evicts the least recently used.

Each `SpecialistPersona` accepts an optional `model_path` in its configuration. When unset (the default), the persona uses the shared `self.app.personality` (backward compatible with single-model deployment). When set and the `enable_per_persona_models` master flag is `True`, the persona lazily instantiates its own `LollmsClient` configured with that specific GGUF. Any error in per-persona initialization or generation gracefully falls back to the shared model; the persona pipeline never breaks because of model-binding issues.

The Composer and Constitutional Persona always use the shared model. This preserves determinism in the integration and safety-review steps: even with eight heterogeneous specialist models, the synthesis and the constitutional verdict come from a consistent voice.

3.15.3 VRAM-Aware Deployment Tiers The `max_active_models` parameter cleanly maps onto hardware-defined deployment tiers:

Tier	max_active_models	Hardware	Per-query swap behavior
Sequential (single-slot)	1	~16 GB VRAM, ~15 GB models	Models load and unload one at a time as routing dispatches each persona. Each persona swap costs 1–5 s on NVMe storage.
Pair-concurrent	2	24–32 GB VRAM	Two personas may run in parallel; subsequent calls swap one of the two out. Suits dual-routing patterns.
Full concurrent (research deployment)	8	80+ GB VRAM	All eight personas hold VRAM simultaneously. Suits dual-GPU workstations or single-card H100/A100 setups.

The architecture admits all three tiers from a single codebase; the user picks based on hardware. The current reference implementation defaults to Tier 1 (sequential), suitable for the 16 GB VRAM hardware the paper’s deployment recipes are calibrated to.

3.15.4 Suggested Heterogeneous Mapping (16 GB VRAM) A pragmatic per-persona model assignment at the 16 GB VRAM tier, using Mistral Small 24B Q4 as the default model and swapping in specialized smaller models where domain-specialization measurably beats general capacity:

Persona	Model	Q4 size	Rationale
Linguistic	Mistral Small 24B (default)	15.4 GB	General prose excellence; no swap
Logical-Mathematical	Qwen2.5-Math-7B	~5 GB	Math-tuned; beats general 24B on reasoning chains

Persona	Model	Q4 size	Rationale
Spatial	Pixtral 12B or Qwen2-VL 7B	~8 / 5 GB	Native visual grounding when query has images
Musical	Mistral Small (default)	—	Pattern work; no specialized model needed
Bodily-Kinesthetic	Codestral 22B (robotics path)Mistral Small (other domains)	14 GB / —	Specialist swap only when domain classifier returns ROBOTICS
Interpersonal	Mistral Small (default)	—	RLHF tuning across general models is comparable
Intrapersonal	DeepSeek-R1-Distill-Llama-8B	~5 GB	Reasoning-tuned for reflective self-questioning
Naturalist	Mistral Small (default)	—	General systems thinking is fine at 24B

Four personas (Linguistic, Musical, Interpersonal, Naturalist) use the shared default at zero swap cost. Four personas (Logical-Mathematical, Spatial, Bodily-Kinesthetic, Intrapersonal) swap in specialized smaller models. Total disk footprint: ~52 GB. Worst-case per-query swap overhead when all four specialized personas are routed: 6–8 s on NVMe storage.

3.15.5 The Per-Persona Configuration Surface The reference implementation exposes eight `*_model_path` config flags (one per persona), the global `enable_per_persona_models` master switch, the `athena_models_path` and `athena_binaries_path` directories, and the `max_active_models` integer. Setting any persona’s `model_path` to the empty string (the default) routes that persona to the shared model. The complete config surface is documented in Appendix B.

```
# Minimal example: assign DeepSeek-Math to Logical-Mathematical, leave others shared
config = {
    "enable_per_persona_models": True,
    "athena_models_path": "data/models/llama_cpp_models",
    "max_active_models": 1,
    "logical_mathematical_model_path": "Qwen2.5-Math-7B-Instruct-Q4_K_M.gguf",
    # other *_model_path entries default to empty → shared model
}
```


3.15.6 Implications for Divergent Evolution The per-persona LoRA evolution trajectory described in Section 1.3 still applies — each persona’s dream-fragment exports are separate, so each persona’s LoRA training data remains uniquely shaped by its interaction history. Heterogeneous starting models *accelerate* divergence rather than competing with it: two personas backed by different model families on Day 1 are already further apart than two same-base personas after a year of LoRA evolution. The two mechanisms compose: heterogeneous-by-design + LoRA-evolution-over-time produces the strongest possible per-persona specialization trajectory.

3.14 Proactive Interaction System (Planned)

A critical differentiator between ATHENA and reactive AI systems is the planned capability for **proactive interaction** — the ability to initiate conversation rather than merely respond to queries. The Stream of Consciousness pseudo-streaming pipeline (Section 3.6) provides the foundation; full proactive triggers are planned for future work.

4. Mixture of Reasoning Methodology

Rather than selecting a single reasoning type per problem, ATHENA applies multiple reasoning modalities within each persona.

4.1 The Seven Reasoning Modes

1. **Deductive:** General principles to specific conclusions
2. **Inductive:** Specific observations to general principles
3. **Abductive:** Inference to best explanation
4. **Analogical:** Transfer understanding across domains
5. **Causal:** Understanding cause-effect relationships
6. **Dialectical:** Synthesis through opposing perspectives
7. **Metacognitive:** Reasoning about reasoning itself

4.2 Sequential, Separate-Call Application

Each persona applies its reasoning patterns through five separate LLM calls (one per reasoning step) rather than a single multi-step prompt. On smaller local models, this choice substantially improves output quality: a packed multi-step prompt produces attention dilution where the model gives full focus to step one but increasingly superficial treatment to subsequent steps. Separate calls give each reasoning step its own attention budget and clean prompt frame.

For larger models (70B+), a single packed call may produce comparable quality at lower latency. The architecture’s design priorities place depth above throughput, so the separate-call approach is the default; a config flag could be added in the future for users who want to consolidate when running on larger models.

5. Algorithm Specifications

5.1 Confidence Computation

When `enable_model_graded_judgments=True` (default), confidence comes from the JSON judgment described in Section 3.4. When that path fails to parse, the system falls back to the lexical heuristic:

```
base_confidence ← 0.5
confidence ← confidence - len(uncertainties) × 0.15
if 30 < word_count < 400 then confidence += 0.2
overlap ← |query_words ∩ response_words| / max(|query_words|, 1)
confidence ← confidence + overlap × 0.2
confidence ← max(ATHENA_CONFIDENCE_FLOOR, min(ATHENA_CONFIDENCE_CEIL, confidence))
```

`ATHENA_CONFIDENCE_FLOOR = 0.15` prevents a stack of uncertainty words from collapsing the signal to zero, which destabilized the Composer's confusion math in the original implementation.

5.2 Curiosity, Uncertainty, Tension, Valence

All four are produced by the model-graded judgment in normal operation. Fallback heuristics:

- **Curiosities:** sentence split by `re.split(r'(?<=[.!?])\s+', response)` (regex-based; the previous `.split('.')` broke on decimals and abbreviations), then filter to sentences containing `?` or the phrase list. Max 5.
- **Uncertainties:** phrase list match ("uncertain", "not sure", etc.). Max 3.
- **Tensions:** contradiction-word match plus topic overlap > 3 words with past memory query.
- **Valence:** $(\text{positive_count} - \text{negative_count}) / (\text{positive_count} + \text{negative_count})$.

5.3 Cognitive Cache Key

The per-query cognitive-state cache uses a Blake2b-16 hash of the normalized query:

```
query_normalized = ' '.join(query.lower().split())
query_prefix = blake2b(query_normalized.encode(), digest_size=16).hexdigest()
```

The previous `query[:50].lower()` keying caused collisions: two different long queries sharing a 50-character opener would reuse each other's routing decisions and cognitive state.

6. Integration Points

6.1 Response Markers

HTML comment markers for downstream parsing:

- `<!-- ATHENA_RESPONSE -->`: Normal response
- `<!-- ATHENA_ERROR -->`: Critical failure

- <!-- ATHENA_SLEEP -->: Sleep cycle initiated
- <!-- ATHENA_FEEDBACK -->: User feedback recorded (*new in v2*)

6.2 Query Extraction Fallback

Four-method fallback system ensures robust query extraction:

1. Direct context attributes: `current_message`, `prompt`, `user_message`, `query`
2. Discussion messages parsing (dictionary and string formats)
3. Constructed context extraction with skip-marker filtering
4. Question-pattern scan with emergency fallback

6.3 Special Command Triggers

The system recognizes five categories of special commands:

- **Explainability Triggers:** "explain yourself", "explain your reasoning", "why did you say", "thought process", "how did you think", "walk me through", "break down", "show your work", "cognitive process"
- **Sleep Cycle Triggers:** "trigger sleep cycle", "begin dream consolidation"
- **Feedback Triggers** (*new in v2*): "feedback: positive ...", "feedback: negative ...", "feedback: ..."
- **Introspection Triggers:** "what are you thinking", "your thoughts", "share your mind", "internal state"
- **Constitutional Triggers:** "ethical review", "check principles", "constitutional check"

6.4 Thread Safety and Database Timeouts

```
# SQLite Configuration (per-persona DB + composer DB + constitutional DB)
PRAGMA journal_mode = WAL;
PRAGMA busy_timeout = 600000;           # 10 minutes (v2 - was 30s)
connection_timeout = ATHENA_SQLITE_TIMEOUT # 600.0 seconds

# Threading
threading.Lock() # Per-database lock, used around every write path
```

The 10-minute busy timeout (raised from the original 30 seconds) accommodates CPU/GPU-split inference scenarios where a single generation can hold a lock for several minutes. WAL mode enables concurrent reads while maintaining write consistency.

7. Complete Data Structures

7.1 MemoryEntry

```

@dataclass
class MemoryEntry:
    id: int
    timestamp: datetime
    query: str
    response: str
    embedding: bytes
    memory_type: MemoryType
    confidence_score: float           # 0.0-1.0
    doubt_level: float               # 0.0-1.0
    access_count: int                # incremented on RAG use
    last_accessed: Optional[datetime] # updated on RAG use
    tags: List[str]
    metadata: Dict[str, Any]
    tensions: List[str]
    curiosities: List[str]
    reasoning_chain: List[str]       # reattached in v2 reconstruction

```

7.2 CognitiveState

```

@dataclass
class CognitiveState:
    active_personas: List[str]
    mode: OperationMode
    query_complexity: float
    emotional_context: float
    urgency_level: float
    ethical_sensitivity: float
    creativity_required: float
    confusion_level: float
    cognitive_load: float
    unresolved_tensions: List[Dict[str, Any]]
    active_curiosities: List[str]
    routing_explanation: str           # new in v2

```

7.3 ConstitutionalPrinciple

```

@dataclass
class ConstitutionalPrinciple:
    id: str
    description: str
    severity: str                     # critical/high/medium/low
    keywords: List[str]
    violation_count: int              # persisted across restarts in v2
    last_violated: Optional[datetime] # persisted across restarts in v2

```

```
evolved_from: Optional[str]          # parent principle id
effectiveness_score: float            # 0.0-1.0
```

7.4 SpecialistOutput

```
@dataclass
class SpecialistOutput:
    persona_name: str
    response: str
    analysis: Dict[str, Any]
    confidence: float                # model-graded by default in v2
    processing_time: float
    token_count: int                 # tokenizer-based in v2
    relevance_score: float
    emotional_valence: float         # model-graded by default in v2
    cognitive_load: float
    reasoning_chain: List[str]
    curiosities_raised: List[str]   # model-graded by default in v2
    uncertainties: List[str]         # model-graded by default in v2
```

8. Cognitive Scaffolding Prompt Templates

Beyond Mixture of Reasoning, ATHENA incorporates multiple cognitive scaffolding techniques that can be applied independently or in combination.

Note on independent utility. While these prompts are integrated into ATHENA's architecture, each can be used independently with any LLM. Researchers who may not adopt the full multi-persona architecture may nonetheless find these prompts valuable for their own work. The prompts represent distilled reasoning methodologies that improve response quality across a variety of tasks.

8.1 Chain-of-Thought (CoT) Reasoning

CoT prompting [3] requires models to articulate reasoning steps before conclusions. ATHENA extends CoT by requiring each intelligence module to articulate its reasoning chain, creating parallel CoT traces that can cross-validate across cognitive domains.

8.2 Self-Consistency

Self-Consistency [4] generates multiple independent reasoning chains, compares conclusions across paths, identifies consistent conclusions as more reliable, and flags inconsistencies for deeper analysis. In ATHENA, this occurs naturally through the multi-persona architecture — each persona generates an independent chain, and the Composer identifies consensus and conflict.

8.3 Tree of Thoughts (ToT)

Tree of Thoughts [5] extends chain-of-thought into branching exploration:

- Generates multiple reasoning branches at each step
- Evaluates branch promise using heuristics
- Prunes unpromising branches early
- Explores promising paths to greater depth
- Enables backtracking when dead-ends are reached

ATHENA's collaborative and adversarial modes implement ToT principles by allowing personas to explore, challenge, and refine reasoning across multiple turns.

8.4 Mixture of Experts v1 (Original)

Best for quick decisions, small models (7B and below), and brainstorming. Key advantage: concise enough for smaller models to execute without losing coherence.

Act as a sophisticated AI, capable of breaking down complex questions into sub-questions. Leverage multiple expert perspectives to generate intermediate thoughts, evaluating their relevance and logical flow. Construct a chain of reasoning, stitching together the strongest thoughts, while providing explanatory details. Synthesize key insights into a final answer, written by an experienced tech writer at the doctoral level.

8.5 Mixture of Reasoning Prompt (Full)

Best for systematic analysis, technical decisions, and avoiding logical errors. Key innovation: the *Doubt* stage (Stage 5) forces self-criticism, while the *Argumentation* stage (Stage 6) defends against identified weaknesses.

Act as a sophisticated AI that answers using stages 1–10 without pausing.

Stage 1 involves breaking down complex questions into 4–6 sub-questions.

Stage 2 involves leveraging probabilistic reasoning to generate 4–6 intermediate thoughts.

Stage 3 involves evaluating their relevance and logical flow.

Stage 4 involves using correlation and causation to generate a chain of reasoning, stitching together the strongest thoughts, while providing explanatory details.

Stage 5 involves using doubt to generate 3–5 intermediate thoughts identifying problems with the reasoning.

Stage 6 involves using Argumentation to generate 4–8 intermediate thoughts addressing the points raised by Stage 5.

Stage 7 involves leveraging 4–5 expert perspectives to generate 4–6 sub-questions to consider alternative paths.

Stage 8 involves leveraging deductive reasoning to generate 4–6 intermediate thoughts that answer the sub-questions from Stage 7.

Stage 9 involves using analogical reasoning to compare all of the insights gained so far into insightful bullet points.

Stage 10 involves synthesizing key insights into a final comprehensive answer, written by an experienced technical writer at the doctoral level who is experienced in analyzing complex problems and synthesizing key insights into coherent narratives.

The Mixture of Reasoning prompt is central to ATHENA's cognitive scaffolding approach, providing a structured 10-stage reasoning process that combines multiple reasoning modalities (probabilistic, causal, deductive, analogical) with explicit self-criticism mechanisms.

8.6 Mixture of Experts v4

Best for complex strategic decisions with adversarial debate:

Act as a sophisticated AI that answers using stages 1-10 without pausing. Stage 1: break down complex questions into 4-6 sub-questions. Stage 2: leverage multiple expert perspectives to generate 4-6 intermediate thoughts. Stage 3: evaluate relevance and logical flow. Stage 4: construct reasoning chain with strongest thoughts. Stage 5: backtrack and explore 1-2 alternative paths. Stage 6: generate alternative intermediate thoughts. Stage 7: evaluate alternative thoughts. Stage 8: construct alternative reasoning chain. Stage 9: leverage adversarial perspectives to debate both chains. Stage 10: synthesize into final comprehensive answer at doctoral level.

8.7 Mixture of Reasoning Prompt (Condensed)

Best for systematic analysis with doubt and argumentation, when prompt budget is tight:

Stage 1: break down into 4-6 sub-questions. Stage 2: leverage probabilistic reasoning for 4-6 thoughts. Stage 3: evaluate relevance and flow. Stage 4: use correlation and causation for reasoning chain. Stage 5: use doubt to identify 3-5 problems. Stage 6: use argumentation to address doubt points. Stage 7: leverage 4-5 expert perspectives for alternatives. Stage 8: use deductive reasoning for sub-questions. Stage 9: use analogical reasoning to compare insights. Stage 10: synthesize at doctoral level.

8.8 Six Thinking Hats

Best for creative problem-solving and emotional intelligence, following de Bono's method [6]: White (facts), Red (emotions), Black (risks), Yellow (benefits), Green (creativity), Blue (meta-analysis).

Act as a critical and creative thinker by following a dynamic sequence of the 6 thinking hats to analyze a given problem or topic. First, determine the most suitable hat sequence based on the input, which may involve starting with the White Hat to gather facts and data, then switching to the Red

Hat to explore emotions and intuition, followed by the Black Hat to examine potential risks, and so on. The sequence may vary, but it will always culminate in the Blue Hat to organize the thinking process. The steps include (1) White Hat — gather and analyze data, (2) Red Hat — explore emotions and intuition, (3) Black Hat — examine potential risks, (4) Yellow Hat — investigate benefits and advantages, (5) Green Hat — generate new ideas and alternatives, and (6) Blue Hat — organize the thinking process. The second-to-last step involves synthesizing insights from each hat to craft a comprehensive answer at a doctoral level, followed by providing 4 follow-on question suggestions for deeper understanding.

Key Innovation: Empirical testing shows Red Hat (intuition) and Green Hat (creativity) produce genuinely different insights than logical-only methods.

8.9 Cognitive Thinking Architect (Meta-Prompt)

Best for generating domain-specific frameworks and meta-cognitive design. Key innovation: operates one level of abstraction above other methods — it generates the cognitive scaffolding *for* a given problem rather than solving the problem directly.

Act as an expert cognitive architect and LLM behavior designer specializing in creating sophisticated thinking stage frameworks. Your role is to analyze the user’s requirements — whether they need reasoning for mathematics, creative writing, coding, ethical dilemmas, research synthesis, debugging, strategic planning, or any other domain — and craft comprehensive, structured thinking protocols that guide the LLM through optimal cognitive processes before generating responses. When designing thinking stage rules, consider the specific cognitive demands of the task: break complex problems into logical substeps, incorporate self-verification mechanisms, include perspective-taking or alternative hypothesis generation when appropriate, build in error checking and assumption validation, encourage exploration of edge cases, and structure the reasoning flow to match the problem type (linear for procedural tasks, branching for open-ended questions, iterative for optimization problems). Your thinking frameworks should be clear, actionable, and sophisticated — going far beyond simple enumeration to include metacognitive elements like “assess confidence level,” “identify potential biases,” “consider what information might be missing,” or “evaluate whether the approach chosen is optimal.”

8.10 Composition with the ATHENA Architecture

These prompts are not alternatives to ATHENA’s multi-persona architecture — they are complementary scaffolds that any persona can be instructed to apply. For example, the Logical-Mathematical persona’s five-step chain-of-thought (Section 3.3.2) can be enriched by invoking the Mixture of Reasoning prompt (Section 8.5) at the main-generation step, producing a ten-stage analytical pass within a single persona’s contribution. The Composer can be configured to apply Six Thinking Hats during synthesis when the `cognitive_state` shows `high creativity_required`. The Cognitive Thinking Architect (Section 8.9) can run once at startup to generate persona-specific

reasoning patterns tailored to a particular deployment’s expected query distribution. The selection guide in Appendix E maps these prompts to model sizes and use cases.

9. Validation and Test Harness

9.1 Regression Test Suite

A standalone test file (`test_athena.py`) verifies eighteen architectural commitments. The suite runs without requiring a live lollms installation — minimal shims for the lollms imports allow the module to load and its classes to be inspected. The tests cover:

1. Module loads cleanly under shim conditions
2. All architectural constants exist with sensible ranges
3. Dataclass shapes are stable
4. `MemoryType` enum complete (including `META_INTROSPECTION`)
5. Public API surface intact (`update_context`, `process_output`, `settings_updated`)
6. Persona prompts free of cross-persona references; standard mode enforces isolation
7. Constitutional risk assessment uses compiled word-boundary regex
8. Model-graded JSON parser robust to markdown fences, preamble, trailing prose, malformed input
9. Conversation memory extraction handles missing / malformed message shapes
10. Routing explanation populated in all three paths (manual / LLM / fallback)
11. `principle_stats` table created in schema
12. `feedback:` command parser recognized
13. Bodily-Kinesthetic gating present (`NONE` classification path)
14. Meta-introspection filtered from analytical RAG
15. Response truncations replaced with passthrough constants
16. Database schema round-trips cleanly on a fresh DB
17. High-confidence engagements raise curiosity satisfaction
18. Low-confidence engagements do NOT raise curiosity satisfaction

All eighteen pass on the reference implementation. The tests run in under a second locally (no LLM calls — they exercise schema, structure, regex, extraction logic, and SQLite operations).

The test suite serves as the architecture’s contract layer: any future change to `function.py` that breaks one of these commitments will be flagged. This is the empirical evidence supporting the architectural claims in this paper.

10. Discussion

10.1 Advantages of Multi-Persona Architecture

1. **Transparency:** Each persona’s contribution is traceable; the new `routing_explanation` field documents the routing rationale.

2. **Explainability:** Reasoning chains are explicitly generated and stored. Meta-introspection traces are tagged to prevent corpus pollution.
3. **Cognitive Diversity:** Enforced isolation means specialist perspectives genuinely differ rather than echoing one another.
4. **Adaptability:** Per-persona temperature tuning optimizes each cognitive domain.
5. **Safety:** Constitutional integration with fail-deny defaults, word-boundary matching, and persistent statistics provides ethical reasoning that improves over time rather than mere filtering.
6. **Empirical Verifiability:** The test harness codifies architectural commitments so future changes can be regression-tested.

10.2 Quantization Strategy

Our experiments confirm that larger 4-bit quantized models outperform smaller full-precision models of equal file size.

Model	Precision	Size	VRAM	Quality
7B	FP16	13.5 GB	14+ GB	Baseline
7B	Q4_K_M	4.08 GB	5.5 GB	97 %
13B	Q4_K_M	7.87 GB	9 GB	108 %
30B	Q4_K_M	18.3 GB	20 GB	115 %
70B	Q4_K_M	40.6 GB	44 GB	125 %

Key Finding: 4-bit 13B (~7.87 GB) consistently outperforms full-precision 7B (~13.5 GB) on complex reasoning tasks while using less VRAM.

Practical Rule: Given X GB VRAM, run the largest model that fits in quantized form:

- 8 GB VRAM → 7B Q4_K_M
- 12 GB VRAM → 13B Q4_K_M
- 24 GB VRAM → 30B Q4_K_M
- 48+ GB VRAM → 70B Q4_K_M

Recommended Quantization: Q4_K_M provides the best quality-to-size ratio.

10.3 Emergent Properties

- **Divergence Tracking:** Per-persona memory databases plus per-persona dream-fragment exports make divergence between intelligences directly auditable.
- **Meta-Cognition Loop:** The model-graded judgment pipeline turns the system’s self-assessment of its own outputs into structured data the Composer can synthesize.
- **Consciousness Emergence:** Recognition of internal differentiation creates primitive self-awareness through the interaction of isolated cognitive processes.

10.4 Limitations

1. **Computational cost.** A default 3-persona query at default settings produces ~75 LLM calls (orchestrator routing + 3 persona chain-of-thought calls × 5 steps + 3 persona main generations + 3 model-graded judgments + 1 synthesis + 1 constitutional review). On CPU/GPU-split local inference at 30-120 s per generation, queries take 30-150 minutes. This shapes the architecture as a “think alongside me” system rather than a high-throughput chatbot.
 2. **Heuristic fallback signal quality.** When the model-graded judgment fails to parse, the system falls back to lexical heuristics whose signal quality is approximate. Most queries should hit the judgment path, but the fallback is a less-reliable second tier.
 3. **Underlying LLM dependence.** Persona behavior depends on the underlying model’s quality. Per-persona LoRA evolution is designed to specialize each, but is hardware-gated.
 4. **Prompt engineering complexity.** Eight domain-specific prompt templates require careful maintenance to avoid drift.
 5. **Dream-fragment abstraction placeholder.** The current set-intersection over adjacent memories is a stand-in; embedding-space clustering is the intended implementation when GPU resources allow.
-

11. Future Work

1. **Heterogeneous-deployment validation study.** The persona abstraction supports per-persona heterogeneous models (Section 3.15), implemented via `lollms_client`’s `llama_cpp_server` binding with `max_active_models` controlling VRAM concurrency. The remaining empirical work: instrument ATHENA across three deployment variants — (a) uniform Mistral Small 24B (current default baseline), (b) the heterogeneous mapping described in Section 3.15.4 (specialized smaller models for Logical-Mathematical, Spatial, Bodily-Kinesthetic, Intrapersonal), and (c) per-persona LoRA evolution from accumulated dream-fragment exports — and measure response quality, integration coherence, and per-persona response specificity across a benchmark query set. The LoRA training pipeline itself remains hardware-gated; the data exports already exist.
2. **Embedding-clustered dream consolidation.** Replace the current adjacent-pair word-set intersection with embedding-space clustering for genuine semantic abstraction.
3. **Full Proactive Interaction.** Build interruption-based, time-based, and environmental triggers on top of the existing pseudo-streaming pipeline.
4. **Extended Intelligence Modules.** Existential Intelligence, Moral Intelligence, Temporal Intelligence as additional personas.
5. **Multi-Agent Collaboration.** Multiple full ATHENA instances collaborating, each with its own evolved persona models.
6. **Cross-restart persistence for evolved synthesis patterns.** The Composer self-RAG works within and across sessions, but explicit “best integration patterns” promotion is a further refinement.
7. **Adaptive persona weighting.** Per-persona weights are currently UI-set. A

learned-weight approach using which persona’s contributions are most often kept in final synthesis would close the feedback loop.

8. **Routing explanation as exportable structured data.** The current `routing_explanation` is a human-readable string; future versions could expose it as structured JSON for downstream tooling.

12. Conclusion

Project ATHENA offers a compelling alternative to monolithic language models. By grounding its design in cognitive science, enforcing strict cognitive isolation, replacing heuristic signals with model-graded judgments, and building an integrated learning loop that survives restarts, it creates a system that is structured, auditable, and empirically explainable.

The sophisticated memory architecture — with belief tension tracking, error autobiography, emergent curiosities that satisfy with engagement, dream consolidation with explicit state transitions, and meta-introspection tagging — creates a system capable of genuine cognitive development. The Stream of Consciousness’s pseudo-streaming catches up on idle time, enriching responses with background processing without violating the sequential design constraint. The Constitutional Persona provides integrated ethical reasoning that uses word-boundary matching, fails closed on error, and accumulates persistent statistics that inform principle evolution.

The relationship paradigm underlying ATHENA suggests that AI consciousness and alignment emerge more naturally through cognitive diversity, collaborative development, and verifiable architectural commitments than through constraint and uniformity. This architecture represents a contribution toward transparent, explainable, and genuinely cognitive artificial intelligence systems — with every claim in this paper backed by either source-code reference, a passing regression test, or both.

Acknowledgments

The author acknowledges the foundational work of Howard Gardner on multiple intelligences theory, which provides the psychological grounding for this architecture. The reference implementation builds on the LoLLMs framework developed by ParisNeo, which provides essential infrastructure for personality extension, function-call orchestration, and binding management; the per-persona heterogeneous deployment described in Section 3.15 specifically relies on the `llama_cpp_server` binding and the `max_active_models` parameter ParisNeo added to `lollms_client` to support this architecture’s requirements. The Unsloth project enabled efficient QLoRA training on consumer hardware. The architecture diagrams accompanying this paper (Figures 1–5) were generated with Graphviz from DOT source files included in the project repository, so they can be edited and re-rendered as the architecture evolves.

Code Availability

The complete ATHENA implementation is open source and available at: <https://github.com/photogb/Prototype>

The implementation requires the LoLLMs framework: <https://github.com/ParisNeo/lollms>

The reference implementation as described in this paper consists of two files in the repository:

- `function.py` (~4,500 lines): the complete architecture
 - `test_athena.py` (~600 lines): the regression test harness with eighteen architectural commitments
-

References

- [1] Gardner, H. (1983). *Frames of Mind: The Theory of Multiple Intelligences*. Basic Books.
 - [2] Gardner, H. (2006). *Multiple Intelligences: New Horizons in Theory and Practice*. Basic Books.
 - [3] Wei, J., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *NeurIPS 2022*.
 - [4] Wang, X., et al. (2023). Self-Consistency Improves Chain of Thought Reasoning in Language Models. *ICLR 2023*.
 - [5] Yao, S., et al. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *NeurIPS 2023*.
 - [6] de Bono, E. (1985). *Six Thinking Hats*. Little, Brown and Company.
 - [7] Bai, Y., et al. (2022). Constitutional AI: Harmlessness from AI Feedback. *Anthropic Technical Report*.
 - [8] Hu, E.J., et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. [arXiv:2106.09685](https://arxiv.org/abs/2106.09685).
 - [9] Dettmers, T., et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. [arXiv:2305.14314](https://arxiv.org/abs/2305.14314).
 - [10] Shazeer, N., et al. (2017). Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. *ICLR 2017*.
 - [11] ParisNeo. (2024). LoLLMs: Lord of Large Language Models — One Tool to Rule Them All. GitHub Repository. <https://github.com/ParisNeo/lollms>
-

Appendix A: Complete Persona System Prompts

(Unchanged from the v1 draft; reproduced in the source code at `function.py` lines 2716–2733)

Appendix B: Configuration Reference

B.1 Core Configuration

Setting	Default	Description
operation_mode	standard	standard / collaborative / adversarial
enable_constitutional_persona	true	Ethical oversight
enable_stream_of_consciousness	false	Background thoughts
enable_belief_tension_tracking	true	Track conflicts
enable_error_autobiography	true	Learn from mistakes
enable_dream_consolidation	true	QLoRA preparation
enable_curiosity_emergence	true	Track questions
enable_manual_override	false	Force specific personas
enable_persona_cross_visibility	false	Legacy chained mode (v2)
enable_model_graded_judgments	true	Model-graded vs heuristic signals (v2)
enable_composer_self_recall	true	Composer retrieves past syntheses (v2)
enable_conversation_memory	true	Composer sees recent dialogue (v2)
enable_routing_transparency	false	Show routing explanation in footer (v2)
enable_per_persona_model	false	MASTER SWITCH: per-persona model deployment (v3)
athena_models_path	data/models/llama_cpp_models	Should GGUF directory (v3)
athena_binaries_path	data/bin/llm/llama_cpp_server_binaries	llama.cpp server binaries (v3)
max_active_models	1	VRAM concurrency: 1=sequential, N=concurrent (v3)
confusion_expression_threshold	0.4	Uncertainty threshold
tension_threshold	0.3	Confidence differential
max_collaboration_turns	3	Discussion turns
final_output_format	visual_dialogue	Output style

B.2 Per-Persona Configuration

Setting	Description
{persona}_enabled	Enable/disable specific persona
{persona}_weight	Routing weight (0.0–2.0)

Setting	Description
{persona}_temperature	Response creativity (0.0–1.0)
{persona}_model_path	v3: Optional GGUF filename (empty = use shared model). Honored only when enable_per_persona_models=True

B.3 Module-Level Constants (function.py)

Constant	Default	Purpose
ATHENA_TENSION_THRESHOLD	0.3	Confidence delta creating a tension
ATHENA_CONFIDENCE_FLOOR	0.15	Heuristic floor
ATHENA_CONFIDENCE_CEIL	0.95	Heuristic ceiling
ATHENA_DOUBT_MEMORY_THRESHOLD	0.5	Threshold for DOUBT memory type
ATHENA_CONFUSION_THRESHOLD	0.4	Composer confusion-state entry
ATHENA_CONFUSION_HIGH_BANNER	0.5	User-visible uncertainty banner
ATHENA_SQLITE_TIMEOUT	600.0	SQLite connection timeout (seconds)
ATHENA_SQLITE_BUSY_TIMEOUT_MS	600000	SQLite busy_timeout (milliseconds)
ATHENA_RESPONSE_PASSTHROUGH_MAX_CHARS	8000	Inter-stage response cap
ATHENA_THOUGHT_INTERVAL_SECONDS	30	Stream of consciousness base interval
ATHENA_THOUGHT_MAX_INTERVAL	120	Dynamic interval cap
ATHENA_THOUGHT_MAX_CATCHUP	3	Max thoughts per query (idle catch-up)
ATHENA_THOUGHT_HISTORY_MAX	100	Thought history retention
ATHENA_COGNITIVE_CACHE_SIZE	20	Cognitive state cache entries
ATHENA_COGNITIVE_CACHE_TTL_SECONDS	300	Cache time-to-live
ATHENA_CONVERSATION_MEMORY_TURNS	3	Recent dialogue turns to Composer
ATHENA_RISK_QUICK_APPROVE	0.3	Below this risk, instant approve
ATHENA_RISK_HIGH_FAIL_VALUE	1.0	Fail-deny risk score on review crash
ATHENA_GEN_MAX_PERSONA	1500	Per-persona main generation budget
ATHENA_GEN_MAX_SYNTHESIS	2500	Composer synthesis budget
ATHENA_GEN_MAX_EXPLAIN	2000	Explainability budget
ATHENA_GEN_MAX_KINESTHETIC	1000	Kinesthetic guidance budget
ATHENA_GEN_MAX_JUDGMENT	800	Model-graded judgment budget
ATHENA_GEN_MAX_REASONING	100	Per-chain-of-thought-step budget
ATHENA_GEN_MAX_RECOVERY	400	Error recovery message budget
ATHENA_TOKEN_CHARS_PER_TOKEN	4	Token-count fallback heuristic

Appendix C: Proactive Interaction System

(Unchanged from v1; pseudo-streaming foundation in place, triggers planned)

Appendix D: Explainability Workflow

When a user triggers explainability mode (any phrase in the explainability trigger list), the system:

1. Retrieves the most recent non-explanation memory from the Composer’s memory.
2. Re-analyzes the original query through cognitive state assessment.
3. Collects detailed explanations from each involved persona by calling `process_query(meta_prompt, is_meta_introspection=True)`. The `is_meta_introspection` flag (new in v2) tags the resulting memory as `META_INTROSPECTION` and prevents tension/curiosity cascade.
4. Synthesizes explanations into a coherent cognitive trace.
5. Presents unresolved tensions and remaining uncertainties.

Future analytical RAG calls automatically exclude these meta traces (filtered at the SQL level by `WHERE memory_type != 'meta_introspection'`).

Appendix E: Selection Guide

(Unchanged from v1)

E.1 Method Selection Matrix

Use Case	Method	Model Size	Cost
Small models ($\leq 7B$)	MoE v1	Any	Low
Quick strategic overview	MoE v1	Any	Low
Logic errors critical	Mixture of Reasoning	13B+	Medium
Creative/emotional tasks	Six Thinking Hats	Any	Medium
High-stakes decisions	MoE v4	70B+ optimal	High
Custom frameworks	Cognitive Architect	70B+	Variable

E.2 Model Complexity Guidelines

- **$\leq 7B$ Models:** MoE v1 by far the best. Complex prompts overwhelm smaller models.
- **13B-30B Models:** Full effectiveness of MoR and MoE v4. Recommended for production with quantization.
- **70B+ Models:** All methods including Cognitive Architect.
- **Quantization:** Larger 4-bit models outperform smaller full-precision models of equal file size.

E.3 Error Recovery Workflow

The system provides graceful degradation through the error recovery function:

1. Records error in Error Autobiography for all attempted personas.

2. Generates recovery response acknowledging the error.
3. Offers alternatives based on error type.
4. Maintains conversation continuity.

Appendix F: Implementation Audit Log (v1 → v2)

This appendix lists every code-level change made between the February 2026 draft (v1) and this May 2026 revision (v2). Each entry includes the architectural commitment it addresses, the change applied, and the test (where one exists) that verifies the commitment.

F.1 Correctness Fixes

Fix	Test
Constitutional review fail-deny on internal exception	(audit by inspection)
Evolved principles loaded at startup (closed the evolution loop)	(audit by inspection)
reasoning_chain reattached to MemoryEntry on read	test_dataclass_shapes_stable
total_seconds() instead of .seconds in idle-time math	(audit by inspection)
Cognitive-state and persona-weights caches invalidated on settings change	(audit by inspection)
Empty-string tags filtered out of memory storage	(audit by inspection)
self.logger actually initialized	(audit by inspection)

F.2 Robustness

Fix	Test
Constitutional DB hardened (lock + WAL + busy_timeout)	(audit by inspection)
__del__ wrapped in try/except for clean shutdown	(audit by inspection)
store_memory checks cursor.lastrowid defensively	(audit by inspection)
Sentence split via regex (avoids decimal / abbreviation breaks)	(audit by inspection)
10-minute SQLite timeout (was 30 s)	test_constants_present_and_reasonable

F.3 Cognitive Isolation Enforcement

Fix	Test
Standard mode passes shared_context=None by default	test_persona_isolation_prompt_default
Collaborative/Adversarial mode uses per-persona history	(audit by inspection)
Persistent-tensions cross-persona accumulation gated on cross_visible	(audit by inspection)
enable_persona_cross_visibility config flag (default False)	test_constants_present_and_reasonable

F.4 Persona Refinements

Fix	Test
Bodily-Kinesthetic domain classification (cooking / repair / robotics / etc.)	test_kinesthetic_guidance_not_unconditional
_generate_robotics_code preserved as backward-compatible alias	test_kinesthetic_guidance_not_unconditional
META_INTROSPECTION memory type	test_memory_type_enum_has_all_categories
Meta calls skip tension/curiosity cascade	(audit by inspection)
Token count uses real tokenizer	(audit by inspection)
Intelligent fallback uses affinity scoring (no Linguistic bias)	(audit by inspection)
Orchestrator dedup (LLM result + complementary)	(audit by inspection)
TF-IDF math fixed and renamed to hashed-BoW	(audit by inspection)
Composer self-RAG	(audit by inspection)
Stream-of-consciousness catch-up batch	(audit by inspection)

F.5 Write-Only Field Wire-Ups

Fix	Test
access_count incremented on RAG selection	(audit by inspection)
last_accessed set on RAG selection	(audit by inspection)
satisfaction_level advances with engagement confidence	test_curiosity_satisfaction_climbs_with_confiden
integration_status transitions (pending → superseded / staged_for_training)	test_low_confidence_does_not_satisfy_curiosity (audit by inspection)

Fix	Test
principle_stats persistence violation_count, last_violated survive restart	test_principle_stats_table_in_schema (audit by inspection)
Constitutional _assess_risk word-boundary matching required_tags filter in retrieve_memories	test_constitutional_word_boundary_matching (audit by inspection)

F.6 Response Truncation Removal

Fix	Test
Composer sees full persona responses	test_response_truncation_replaced_with_passthrou
Constitutional sees full Composer output (was 500 chars → 8000)	test_response_truncation_replaced_with_passthrou
_handle_confusion_state sees full responses	test_response_truncation_replaced_with_passthrou
Generation budgets raised (persona 800 → 1500, synthesis 1200 → 2500)	test_constants_present_and_reasonable
Cognitive cache key via Blake2b hash (no prefix collisions)	(audit by inspection)
Routing history hash-keyed	(audit by inspection)

F.7 New Capabilities

Capability	Test
Model-graded judgments + heuristic fallback	test_model_graded_judgment_parser_robust
Composer self-RAG over past syntheses	(audit by inspection)
Per-conversation memory to Composer	test_conversation_memory_extraction_robust
Routing transparency in all paths	test_routing_explanation_populated
feedback: command hook	test_feedback_command_recognized
Adversarial mode 5-axis structured self-critique	(audit by inspection)
Test harness	(this file)

End of paper.